



INSTITUTO  
UNIVERSITÁRIO  
DE LISBOA

---

## **Aprendizagem por Reforço para Controlo de Enxames**

Gonçalo Santos

Mestrado em Inteligência Artificial

Orientador:

Doutor Luís Nunes, Professor do Departamento de Ciências e  
Tecnologias da Informação,  
Iscte – Instituto Universitário de Lisboa

2025

[ Página intencionalmente deixada em branco. ]



TECHNOLOGY  
AND ARCHITECTURE

---

Departamento de Ciências e Tecnologias da Informação

## **Aprendizagem por Reforço para Controlo de Enxames**

Gonçalo Santos

Mestrado em Inteligência Artificial

Orientador:

Doutor Luís Nunes, Professor do Departamento de Ciências e  
Tecnologias da Informação,  
Iscte – Instituto Universitário de Lisboa

2025

[ Página intencionalmente deixada em branco. ]

*Aos meus pais e a todos que apoiaram este percurso.*

[ Página intencionalmente deixada em branco. ]

## Agradecimentos

Gostaria de expressar a minha gratidão aos meus familiares, por me terem apoiado ao longo desta jornada académica, e à minha namorada, por ter estado sempre ao meu lado nos bons e nos maus momentos. Agradeço também aos colegas e amigos que fiz ao longo do tempo, bem como aos professores, por terem dado o seu melhor para nos ensinar. Por fim, agradeço à TAISCTE por ter feito parte deste meu percurso de mestrado.

This work was partially supported by national funds through Fundação para a Ciência e a Tecnologia, I.P. (FCT), ISTAR-Iscte Pluriannual Projects UID/04466/2025 (<https://doi.org/10.54499/UID/04466/2025>).

[ Página intencionalmente deixada em branco. ]

## Resumo

Esta dissertação apresenta um estudo comparativo rigoroso entre a Aprendizagem por Reforço Multiagente (MARL) e a Otimização Bio-inspirada para o controlo de enxames robóticos. Através de um simulador de alta fidelidade, implementou-se o *framework* MARL RS2C (com os algoritmos PPO e SAC, sob partilha de parâmetros) e um controlador bio-inspirado neuroevolutivo assente numa rede neuronal de grafos com atenção — responsável pela invariância à dimensão do enxame —, avaliando-se ambos em seis cenários de dificuldade crescente sob tratamento estatístico. Os resultados revelam um *trade-off* central: os métodos de gradiente dominam no cumprimento da tarefa (o SAC resolve os seis cenários), enquanto apenas a arquitetura de atenção sobre grafo permite a transferência *Zero-Shot* para enxames de dimensão não vista ( $N$  de 10 a 100) sem re-treino. A hipótese de que a inteligência adaptativa supera a robustez estática confirma-se, assim, parcialmente: a vantagem reside na *representação* (grafo) e não no algoritmo de optimização.

PALAVRAS CHAVE: Robótica de Enxame, MARL, Otimização Bio-inspirada, GNNs.

[ Página intencionalmente deixada em branco. ]

## Abstract

This dissertation presents a rigorous comparative study between Multi-Agent Reinforcement Learning (MARL) and Bio-inspired Optimization for the control of robotic swarms. Using a high-fidelity simulator, we implemented the MARL framework RS2C (with the PPO and SAC algorithms, under parameter sharing) and a neuroevolutionary bio-inspired controller built on a graph neural network with attention — responsible for invariance to swarm size —, evaluating both across six scenarios of increasing difficulty under statistical treatment. The results reveal a central trade-off: gradient-based methods dominate in task completion (SAC solves all six scenarios), whereas only the graph-attention architecture enables Zero-Shot transfer to unseen swarm sizes ( $N$  from 10 to 100) without retraining. The hypothesis that adaptive intelligence surpasses static robustness is therefore partially confirmed: the advantage lies in the *representation* (graph) rather than in the optimization algorithm.

KEYWORDS: Swarm Robotics, MARL, Bio-inspired Optimization, GNNs.

[ Página intencionalmente deixada em branco. ]

# Índice

|                                                               |      |
|---------------------------------------------------------------|------|
| Agradecimentos                                                | iii  |
| Resumo                                                        | v    |
| Abstract                                                      | vii  |
| Lista de Figuras                                              | xi   |
| Lista de Acrónimos                                            | xiii |
| Capítulo 1. Introdução                                        | 1    |
| 1.1. Enquadramento e Motivação                                | 1    |
| 1.2. Definição do Problema e Justificação da Tese             | 2    |
| 1.3. Objetivos da Tese                                        | 3    |
| Capítulo 2. Fundamentos Teóricos e Tecnologias                | 5    |
| 2.1. Introdução ao Capítulo                                   | 5    |
| 2.2. Swarm Intelligence (SI) e Robótica de Enxame             | 6    |
| 2.3. Paradigmas de Controlo para Enxames                      | 6    |
| 2.4. Abordagem 1: Otimização Bio-inspirada                    | 7    |
| 2.5. Abordagem 2: Multi-Agent Reinforcement Learning (MARL)   | 9    |
| Capítulo 3. Estado da Arte                                    | 13   |
| 3.1. Introdução ao Capítulo                                   | 13   |
| 3.2. Metodologia da Revisão Sistemática                       | 13   |
| 3.3. Análise dos Melhores Resultados                          | 16   |
| 3.4. O Desafio do <i>Deployment Gap</i> em Robótica de Enxame | 17   |
| 3.5. Lacunas Identificadas e Oportunidade de Melhoria         | 19   |
| 3.6. Artigos-Chave e Ligação ao Problema                      | 20   |
| 3.7. Conclusão Parcial                                        | 20   |
| 3.8. Síntese Comparativa da Literatura                        | 21   |
| Capítulo 4. Desenvolvimento do Ambiente de Simulação          | 23   |
| 4.1. Arquitetura de Software e Stack Tecnológica              | 23   |
| 4.2. O Ambiente de Simulação                                  | 24   |
| 4.3. Design da Função de Recompensa                           | 29   |
| 4.4. Desafios de Engenharia e Otimização Computacional        | 30   |
| Capítulo 5. Arquitetura Algorítmica e Controladores           | 33   |

|             |                                                             |    |
|-------------|-------------------------------------------------------------|----|
| 5.1.        | Algoritmo 1: Multi-Agent Reinforcement Learning (RS2C)      | 33 |
| 5.2.        | Algoritmo 2: Otimização Bio-inspirada (Benchmark)           | 34 |
| 5.3.        | Plano de Experiências e Tratamento Estatístico              | 35 |
| Capítulo 6. | Resultados Experimentais e Discussão                        | 37 |
| 6.1.        | Metodologia de Avaliação e Notas de Leitura                 | 37 |
| 6.2.        | Desempenho de Tarefa por Cenário ( $P_{task}$ )             | 37 |
| 6.3.        | Análise Qualitativa da Navegação: Mapas de Calor            | 39 |
| 6.4.        | Convergência e Desempenho no Cenário Base (Sandbox)         | 40 |
| 6.5.        | Desempenho Comparativo por Cenário (Curvas de Aprendizagem) | 41 |
| 6.6.        | Fiabilidade e Variância entre <i>Runs</i> (Boxplots)        | 44 |
| 6.7.        | Visão Global por Algoritmo                                  | 46 |
| 6.8.        | Avaliação Determinística (Tarefa Pura)                      | 48 |
| 6.9.        | Análise de Cenários Complexos e Comportamento Emergente     | 48 |
| 6.10.       | Análise de Escalabilidade (Zero-Shot Transfer)              | 50 |
| 6.11.       | Resiliência e Robustez a Falhas de Agentes                  | 51 |
| 6.12.       | Desempenho Computacional do Simulador                       | 52 |
| 6.13.       | Discussão Global e Validação da Hipótese                    | 52 |
| Capítulo 7. | Conclusões e Trabalhos Futuros                              | 57 |
| 7.1.        | Conclusões                                                  | 57 |
| 7.2.        | Limitações do Estudo                                        | 57 |
| 7.3.        | Contributos Esperados                                       | 58 |
| 7.4.        | Trabalhos Futuros                                           | 59 |
| Referências |                                                             | 61 |
| Apêndice A. | Configurações e Hiperparâmetros                             | 65 |
| Apêndice B. | Excertos de Código Relevantes                               | 67 |
| B.1.        | Mecanismo de Atenção sobre Vizinhos (QKV)                   | 67 |
| B.2.        | Física de Deslizamento em Muros ( <i>Wall-Sliding</i> )     | 67 |
| B.3.        | Campo Geodésico para o <i>Reward Shaping</i>                | 68 |
| B.4.        | Recompensa de Exploração <i>Count-Based</i>                 | 68 |
| Apêndice C. | Recursos Adicionais                                         | 69 |

## Lista de Figuras

|            |                                                                                                                                                                                                                                                                                                                                                                                                    |    |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figura 3.1 | Fluxograma PRISMA 2020: Seleção focada em estudos recentes e quantitativos.                                                                                                                                                                                                                                                                                                                        | 16 |
| Figura 4.1 | Diagrama da arquitetura do simulador e fluxo de interação agente-ambiente.                                                                                                                                                                                                                                                                                                                         | 24 |
| Figura 4.2 | Renderizações 3D dos seis cenários de teste, geradas a partir da geometria real do simulador ( <code>scripts/render_maps.py</code> ). Em cima, da esquerda para a direita: Sandbox, Beco Sem Saída (Muro em U) e Gargalo; em baixo: Quatro Salas, Porta Cooperativa (com a porta destacada a âmbar) e Percepção Cooperativa. A esfera verde assinala o ninho (ou o alvo móvel, no último cenário). | 27 |
| Figura 4.3 | Vista de topo (planta) dos seis cenários, evidenciando a geometria das paredes e a largura das passagens. Mesma ordem da Figura 4.2: em cima, Sandbox, Muro em U e Gargalo; em baixo, Quatro Salas, Porta Cooperativa e Percepção Cooperativa.                                                                                                                                                     | 28 |
| Figura 6.1 | Taxa de sucesso ( $P_{task}$ ) por cenário, em avaliação determinística (30 episódios emparelhados por algoritmo).                                                                                                                                                                                                                                                                                 | 38 |
| Figura 6.2 | Recolhas por episódio (média $\pm$ desvio padrão) por cenário. Mede a magnitude do desempenho, para além do sucesso binário.                                                                                                                                                                                                                                                                       | 39 |
| Figura 6.3 | Potencial de navegação no Muro U: euclidiano (esquerda) vs. geodésico (direita). As curvas de nível indicam a direção para a qual o termo de progresso atrai o agente; só o geodésico contorna a barra do U.                                                                                                                                                                                       | 39 |
| Figura 6.4 | Densidade de ocupação dos robôs no Muro U (GNN, PPO e SAC, da esquerda para a direita). O fluxo que contorna a barra do U pelos lados evidencia a navegação resolvida pelo potencial geodésico.                                                                                                                                                                                                    | 40 |
| Figura 6.5 | Curvas de aprendizagem dos três algoritmos no cenário Sandbox. O eixo das abcissas está normalizado (0–100% do treino). Cada curva corresponde a uma única execução de treino (ver Secção 6.6).                                                                                                                                                                                                    | 41 |
| Figura 6.6 | Curvas de aprendizagem no cenário Beco Sem Saída (Muro em U).                                                                                                                                                                                                                                                                                                                                      | 42 |
| Figura 6.7 | Curvas de aprendizagem no cenário Gargalo.                                                                                                                                                                                                                                                                                                                                                         | 42 |
| Figura 6.8 | Curvas de aprendizagem no cenário Quatro Salas (Labirinto).                                                                                                                                                                                                                                                                                                                                        | 43 |
| Figura 6.9 | Curvas de aprendizagem no cenário Porta Cooperativa.                                                                                                                                                                                                                                                                                                                                               | 43 |

|             |                                                                                                                                |    |
|-------------|--------------------------------------------------------------------------------------------------------------------------------|----|
| Figura 6.10 | Curvas de aprendizagem no cenário Percepção Cooperativa (Alvo Móvel).                                                          | 44 |
| Figura 6.11 | Distribuição dos melhores scores por <i>run</i> : Sandbox (esq.) e Muro U (dir.).                                              | 45 |
| Figura 6.12 | Distribuição dos melhores scores por <i>run</i> : Gargalo (esq.) e Quatro Salas (dir.).                                        | 45 |
| Figura 6.13 | Distribuição dos melhores scores por <i>run</i> : Porta Cooperativa (esq.) e Percepção Cooperativa (dir.).                     | 45 |
| Figura 6.14 | Desempenho do GNN (evolutivo) nos seis cenários.                                                                               | 46 |
| Figura 6.15 | Desempenho do PPO nos seis cenários.                                                                                           | 47 |
| Figura 6.16 | Desempenho do SAC nos seis cenários.                                                                                           | 47 |
| Figura 6.17 | Resumo geral: melhor score médio por algoritmo e cenário (barras de erro = desvio padrão entre <i>runs</i> ).                  | 48 |
| Figura 6.18 | Comportamento emergente no visualizador 3D: contorno do obstáculo em U (esq.) e navegação no labirinto de Quatro Salas (dir.). | 49 |
| Figura 6.19 | Comportamento cooperativo: abertura sincronizada da porta (esq.) e cerco a 360° do alvo móvel (dir.).                          | 50 |
| Figura 6.20 | Degradação (ou estabilidade) do desempenho com o aumento do número de agentes, sem retreino.                                   | 51 |
| Figura 6.21 | Resiliência do exame perante a perda súbita de 10% dos agentes a meio do episódio ( $R_{robust}$ ).                            | 52 |

## Lista de Acrónimos

:

[ Página intencionalmente deixada em branco. ]

# Introdução

### 1.1. Enquadramento e Motivação

A robótica de enxame (*Swarm Robotics*), enquanto subcampo da inteligência artificial e dos sistemas multiagente, representa uma mudança de paradigma fundamental na engenharia contemporânea, transitando de arquiteturas monolíticas e robôs complexos para a coordenação de grandes grupos de agentes simples [1]. Esta transição fundamenta-se na premissa de que a inteligência não reside na complexidade individual de cada membro, mas sim na sofisticação das interações locais entre eles. Esta premissa ecoa a perspectiva de que a inteligência é, na sua essência, um fenómeno de processamento e organização de informação, em larga medida independente do substrato físico que a concretiza [2]. Desde as primeiras conceptualizações em meados da década de 1990 até às aplicações de ponta em 2025, a motivação central deste campo permanece constante: a emulação da inteligência coletiva observada na natureza — como em colónias de formigas ou cardumes — para atingir robustez, flexibilidade e escalabilidade superiores através de comportamento emergente [3], [4].

A urgência no desenvolvimento destes sistemas é impulsionada por aplicações críticas em ambientes de elevada incerteza e perigosidade, tais como a vigilância persistente de infraestruturas, a exploração espacial e a resposta a desastres naturais [5]. Nestes cenários, a redundância intrínseca do enxame é a sua maior vantagem: o sistema é desenhado para ser resiliente a falhas individuais, garantindo que a perda de um único agente não comprometa a integridade da missão global [6].

No entanto, a implementação prática desta visão descentralizada enfrenta barreiras técnicas significativas no que toca ao controlo. Métodos tradicionais, frequentemente dependentes de uma unidade central de processamento ou de modelos matemáticos globais, falham invariavelmente ao escalar o número de agentes ( $N$ ). Este fenómeno, conhecido como a “explosão da complexidade de interações”, ocorre porque o espaço de estados e ações cresce exponencialmente com o aumento da população do enxame, tornando impossível a programação explícita de todas as contingências operacionais [7]. Conforme discutido por Hüttenrauch et al. (2019), a verdadeira escalabilidade exige que as políticas de controlo sejam independentes do número total de agentes, um desafio que as abordagens convencionais ainda têm dificuldade em endereçar de forma eficiente [1].

Neste contexto, a comunidade científica dividiu-se historicamente em duas abordagens principais para o desenho destes controladores descentralizados:

- (1) **Otimização Bio-inspirada:** Métodos como o *Particle Swarm Optimization* (PSO) e Algoritmos Evolutivos (AE) focam-se na robustez *offline* [4]. Tratam o desenho do controlador como um problema de otimização estocástica, procurando os pesos ótimos de uma rede neural ou parâmetros de regras heurísticas antes da implantação real [8], [9]. Embora robustos, estes métodos tendem a ser estáticos, podendo carecer de adaptabilidade perante mudanças dinâmicas e imprevistas no ambiente [10].
- (2) **Multi-Agent Reinforcement Learning (MARL):** Focado na adaptabilidade *online*, o MARL utiliza o paradigma de aprendizagem sequencial para que os agentes descubram políticas cooperativas através da interação direta com o ambiente [11]. A grande promessa do MARL reside na sua capacidade de ajuste em tempo real e na aprendizagem de políticas complexas formalizadas como um *Decentralized Partially Observable Markov Decision Process* (Dec-POMDP) [12].

A pertinência desta investigação sustenta-se na lacuna identificada por autores como Majid et al. (2024), que sublinham a escassez de estudos que comparem diretamente a eficiência e a robustez destas duas escolas em cenários operacionais dinâmicos [7]. Esta tese surge da necessidade urgente de validar qual destas estruturas melhor responde aos desafios de escalabilidade e adaptação exigidos pelas aplicações robóticas de 2025, propondo uma avaliação comparativa rigorosa entre a inteligência adaptativa do MARL e a robustez consolidada da otimização bio-inspirada [13].

## 1.2. Definição do Problema e Justificação da Tese

Apesar do crescimento exponencial no interesse por sistemas autónomos descentralizados, existe uma lacuna crítica na literatura científica: a escassez de uma comparação direta (*benchmarking*) e rigorosa do desempenho de paradigmas concorrentes em cenários de controlo de enxames que sejam, simultaneamente, complexos e dinâmicos [7], [13]. O estado da arte atual revela que, embora existam múltiplos *frameworks* de controlo, a maioria das investigações foca-se no refinamento de um algoritmo isolado, negligenciando a validação cruzada necessária para determinar a aplicabilidade industrial de cada método [14].

O *Multi-Agent Reinforcement Learning* (MARL), embora promissor devido à sua capacidade de aprender comportamentos emergentes complexos, enfrenta desafios teóricos profundos. Entre estes, destaca-se a **não-estacionaridade** do ambiente: como todos os agentes aprendem e alteram as suas políticas simultaneamente, a dinâmica do sistema do ponto de vista de um agente individual muda constantemente, violando a propriedade de Markov necessária para a convergência de algoritmos tradicionais [12]. Além disso, a explosão do espaço de ações conjuntas em enxames de grande escala dificulta a estabilidade do treino e a otimização de funções de recompensa esparsas.

Por outro lado, os métodos evolutivos e bio-inspirados (como o PSO) têm demonstrado uma robustez consolidada, tratando o controlo como a exploração de uma superfície de *fitness* global [4]. Contudo, como notado por Kegeleirs et al. (2025), estas abordagens

resultam frequentemente em controladores estáticos que carecem de adaptabilidade em tempo real perante mudanças estruturais imprevistas no ambiente [15]. Esta rigidez cria o que a literatura denomina como *Deployment Gap* (fosso de implantação), onde algoritmos otimizados em laboratório falham ao enfrentar o ruído e a incerteza do mundo real.

Portanto, a definição do problema desta tese é a **necessidade de uma avaliação comparativa rigorosa e sistemática**. Este estudo visa determinar qual o *framework* de controlo que melhor responde aos desafios práticos do controlo de enxames através de três eixos fundamentais:

- **Desempenho da Tarefa:** Avaliar a eficiência absoluta na concretização de objetivos globais (ex: taxa de cobertura de área ou taxa de recolha em *foraging*). Seguindo a linha de Majid et al. (2024), pretende-se quantificar se a convergência mais lenta do MARL resulta, de facto, numa política final superior em eficiência energética comparativamente aos métodos bio-inspirados [7].
- **Escalabilidade (Zero-Shot Transfer):** Analisar como a *performance* se degrada à medida que o número de agentes ( $N$ ) aumenta para além do cenário de treino. É crucial validar se as arquiteturas baseadas em *Graph Neural Networks* (GNNs), defendidas por Chen et al. (2024), permitem uma escalabilidade superior face à rigidez dos controladores evolutivos fixos [16].
- **Robustez e Adaptação Dinâmica:** Testar a resiliência do sistema perante a perda súbita de agentes (falhas de *hardware*) ou a introdução de obstáculos móveis imprevistos. O objetivo é verificar se a capacidade de aprendizagem contínua do MARL compensa a sua complexidade de implementação face à estabilidade *offline* dos algoritmos evolutivos.

Esta tese justifica-se pela urgência de transformar o controlo de enxames de uma disciplina puramente teórica numa ferramenta de engenharia fiável. Através do uso de um simulador de alta-fidelidade, este trabalho pretende fornecer dados quantitativos que orientem a escolha do paradigma de controlo em função dos requisitos de missão e da disponibilidade de recursos computacionais.

### 1.3. Objetivos da Tese

Com base na definição do problema e na lacuna de investigação identificada na literatura recente, o objetivo geral desta dissertação é **desenvolver um *framework* de simulação de alta-fidelidade e realizar um *benchmarking* comparativo rigoroso entre o *Multi-Agent Reinforcement Learning* (MARL) e algoritmos de otimização bio-inspirados** (e.g., Algoritmos Evolutivos) aplicados ao controlo descentralizado de enxames [15], [17]. Pretende-se determinar, de forma quantitativa, em que condições a inteligência adaptativa e emergente do MARL supera a robustez estática das abordagens de otimização tradicionais [18].

Para atingir o objetivo geral, foram definidos os seguintes objetivos específicos:

- **Desenvolver um Ambiente de Simulação de Referência:** Projetar e implementar um simulador em ambiente Python capaz de modelar tarefas fundamentais de enxame, como o *foraging* (forrageamento) cooperativo dinâmico e a percepção cooperativa (cerco de alvos móveis), em cenários de dificuldade de navegação crescente [13]. O ambiente deve contemplar modelos físicos realistas, observação local e parcial dos agentes e a simulação de falhas súbitas de agentes para testar a resiliência do sistema [6].
- **Implementar um Framework MARL Escalável (RS2C):** Desenvolver o *framework Robust and Scalable Swarm Control* (RS2C) assente em execução descentralizada com partilha de parâmetros [16]. Este objetivo inclui a exploração de **redes de grafos com mecanismos de atenção**, que garantem a invariância à permutação e permitem que um controlador treinado com  $N$  agentes seja aplicado a enxames de outra dimensão sem retreino (*Zero-Shot Transfer*).
- **Implementar uma *Baseline* de Otimização Bio-inspirada:** Desenvolver um controlador baseado em **Neuroevolução** (um Algoritmo Evolutivo, AE) para servir de termo de comparação de desempenho [8], [9]. Este controlador é otimizado *offline*, sem gradientes, para encontrar os pesos de uma rede neuronal de grafos com atenção, representando o paradigma de robustez pré-programada [4].
- **Realizar uma Análise Comparativa e Estatística:** Executar experiências controladas utilizando as métricas de performance da tarefa, escalabilidade e robustez a perturbações [7]. Os resultados deverão ser validados através de tratamento estatístico rigoroso (e.g., testes *t-student*), garantindo que as conclusões sobre a superioridade de um paradigma sobre o outro são cientificamente sustentadas.

## CAPÍTULO 2

### Fundamentos Teóricos e Tecnologias

#### 2.1. Introdução ao Capítulo

Este capítulo estabelece os fundamentos teóricos e os alicerces tecnológicos necessários para uma compreensão aprofundada do controlo descentralizado de enxames. A transição de sistemas robóticos isolados para coletivos coordenados exige uma base conceptual que interliga a biologia, a engenharia de controlo e a aprendizagem automática avançada. Assim, o presente capítulo visa dotar a dissertação do rigor matemático e técnico indispensável para suportar a investigação proposta.

Em primeiro lugar, serão definidos e contextualizados os conceitos de *Swarm Intelligence* (SI) e *Swarm Robotics* (SR). Esta análise inicial é fundamental para compreender como comportamentos globais complexos e auto-organizados podem emergir a partir de regras de interação locais extremamente simples. Conforme discutido por Riviere et al. (2020), a robustez destes sistemas reside precisamente na sua natureza descentralizada, o que impõe desafios únicos ao desenho de controladores eficientes.

Posteriormente, o foco incidirá sobre as duas principais classes de paradigmas de controlo que constituem o núcleo do *benchmarking* desta tese:

- (1) **Otimização Bio-inspirada:** Serão explorados algoritmos como a Otimização por Enxame de Partículas (*Particle Swarm Optimization* - PSO) e os Algoritmos Evolutivos (AE). Estas técnicas tratam o controlo como um problema de busca num espaço de parâmetros global, focando-se na convergência para soluções estáveis e robustas em cenários estáticos.
- (2) **Multi-Agent Reinforcement Learning (MARL):** Será introduzida a formalização matemática do problema através do modelo de Processo de Decisão de Markov Parcialmente Observável Descentralizado (*Dec-POMDP*). Esta secção detalhará como a aprendizagem por reforço permite a emergência de políticas adaptativas através da interação *online*, abordando conceitos críticos como a arquitetura *Centralized Training with Decentralized Execution* (CTDE) e o papel das *Graph Neural Networks* (GNNs) na garantia da escalabilidade do enxame, tal como defendido por Chen et al. (2024).

Ao estabelecer estas definições, este capítulo permite identificar o “Deployment Gap” (fosso de implantação) discutido por Kegeleirs et al. (2025), justificando a necessidade de uma metodologia rigorosa para validar a transição destes algoritmos do domínio da simulação para aplicações reais de alta fidelidade [15].

## 2.2. Swarm Intelligence (SI) e Robótica de Enxame

A Inteligência de Enxame (*Swarm Intelligence* - SI) refere-se ao comportamento coletivo e coordenado de sistemas descentralizados e auto-organizados, sejam eles naturais ou artificiais [3]. Este paradigma baseia-se na observação de sistemas biológicos — tais como colônias de formigas, enxames de abelhas ou cardumes de peixes — onde agentes individuais com capacidades cognitivas limitadas conseguem, através de interações sociais simples, resolver problemas de elevada complexidade que excederiam a capacidade de qualquer indivíduo isolado [4].

A Robótica de Enxame (*Swarm Robotics* - SR) surge como a aplicação direta dos princípios de SI ao domínio de sistemas multi-robô [15]. Ao contrário da robótica cooperativa tradicional, a SR foca-se na coordenação de um grande número de agentes, tipicamente homogêneos e de baixo custo, que operam sem uma infraestrutura de controlo centralizada ou um mapa global do ambiente. O controlo nestes sistemas é inerentemente distribuído, o que confere ao coletivo três propriedades fundamentais identificadas na literatura:

- **Robustez:** A redundância do sistema garante que a missão prossiga mesmo perante a falha de múltiplos agentes [1].
- **Escalabilidade:** O sistema mantém a sua eficiência independentemente da dimensão do enxame ( $N$ ), uma vez que as interações são predominantemente locais.
- **Flexibilidade:** O enxame consegue adaptar-se a diferentes tarefas e ambientes dinâmicos sem necessidade de reprogramação externa.

O alicerce técnico destes sistemas reside no comportamento emergente. Conforme discutido por Riviere et al. (2020), as ações globais coordenadas emergem de um ciclo de *feedback* contínuo entre regras de interação locais simples e o ambiente. Um mecanismo crítico para esta emergência é a **estigmergia**, um método de coordenação indireta onde os agentes alteram o ambiente (por exemplo, através de trilhos de feromonas ou comunicação local ruidosa), influenciando as ações subsequentes de outros membros do grupo.

No entanto, projetar manualmente estas regras locais para atingir um objetivo global específico é um desafio de engenharia não trivial. Como não existe um “conhecimento global” partilhado, o desenho do controlador deve garantir que a soma das micro-decisões individuais resulte na macro-estabilidade do enxame. Este desafio justifica a exploração de métodos de Aprendizagem Automática Avançada, onde, em vez de programar regras explícitas, o sistema “aprende” ou “evolui” as interações ótimas através de reforço ou otimização.

## 2.3. Paradigmas de Controlo para Enxames

O controlo de sistemas multi-robô em larga escala foca-se primordialmente no desenho das chamadas “regras locais” de interação. O desafio fundamental reside na resolução do problema inverso da robótica de enxame: como determinar comportamentos individuais simples que, quando executados em paralelo, resultem numa macro-estabilidade e no

cumprimento de um objetivo global. Existem duas abordagens principais na literatura contemporânea que esta tese se propõe a analisar e comparar:

(1) **Otimização Bio-inspirada (Offline):** Esta categoria abrange duas subfamílias principais frequentemente utilizadas em robótica:

- **Algoritmos Evolutivos (AE):** Inspirados na seleção natural (Darwin), utilizam operadores como mutação e cruzamento para evoluir uma população de controladores ao longo de gerações.
- **Inteligência de Enxame (SI):** Inspirados em comportamentos sociais (e.g., bandos de pássaros), como o *Particle Swarm Optimization* (PSO), onde a solução emerge do movimento cooperativo de partículas no espaço de busca.

Ambas as abordagens procuram *offline* o melhor conjunto de parâmetros fixos, maximizando uma função de *fitness*. Embora gerem comportamentos robustos, resultam frequentemente em controladores rígidos perante mudanças ambientais não previstas.

(2) **Aprendizagem por Reforço (Online/Adaptativa):** ... (o resto mantém-se)

Esta distinção é crucial para o *benchmarking* proposto, pois permite avaliar se o custo computacional e a complexidade de treino da aprendizagem automática são justificados por um ganho real em adaptabilidade face às soluções otimizadas tradicionais.

## 2.4. Abordagem 1: Otimização Bio-inspirada

Esta abordagem utiliza algoritmos de busca meta-heurística para navegar no vasto espaço de parâmetros  $\theta$  de um controlador — frequentemente representado por uma rede neuronal (*Neuroevolution*) ou uma máquina de estados — com o intuito de maximizar uma função de desempenho global  $J(\theta)$ .

### 2.4.1. Fundamentos Matemáticos dos Algoritmos Evolutivos

O processo evolutivo opera sobre uma **população**  $\Pi_g = \{\theta_1, \theta_2, \dots, \theta_K\}$  de  $K$  controladores candidatos, denominados **genomas**, onde cada genoma  $\theta_k \in \mathbb{R}^{|\theta|}$  é o vetor concatenado de todos os pesos e *biases* de uma rede neuronal. A topologia arquitetural da rede é fixada *a priori*; apenas os seus parâmetros numéricos são sujeitos a otimização — esta abordagem denomina-se **Neuroevolução**. Ao contrário do RL, a Neuroevolução não necessita de gradientes calculáveis, tornando-a aplicável a qualquer função de recompensa não-diferenciável.

**Função de Fitness:** O critério de qualidade de cada genoma,  $J(\theta_k)$ , foi deliberadamente desenhado para que a **tarefa** domine a seleção, relegando a recompensa com *shaping* para o papel de gradiente de desempate. Combina assim duas grandezas distintas:

$$J(\theta_k) = \underbrace{\bar{f}_k \cdot \lambda_{task}}_{\text{tarefa}} + \underbrace{\beta \cdot \tanh\left(\frac{\bar{R}_k}{\beta}\right)}_{\text{shaping comprimido}} \quad (2.1)$$

onde  $\bar{f}_k = \frac{1}{E} \sum_{e=1}^E f_{k,e}$  é o número médio de recolhas (a tarefa pura) e  $\bar{R}_k = \frac{1}{E} \sum_{e=1}^E \sum_{t=0}^T \mathcal{R}(s_t, \mathbf{a}_t; \theta_k)$  é o retorno acumulado médio (a recompensa do ambiente no instante  $t$ , com *shaping*;  $T$  é a duração máxima do episódio). O peso  $\lambda_{task} = 10000$  assegura que cada recolha vale mais do que qualquer acumulação de *shaping*, e  $\beta = 5000$  limita o contributo da recompensa. A compressão por tanh é deliberada: ao contrário de um *clip* rígido — que saturaria num valor constante sempre que nenhum genoma recolhe, anulando o gradiente de seleção —, a tanh é monótona e nunca satura abruptamente, mantendo sinal de seleção mesmo nas fases iniciais. Esta formulação distingue *fitness* de recompensa — é possível aumentar a recompensa (rondar a comida acumulando *shaping*) sem aumentar a tarefa —, eliminando o *reward hacking*. A média sobre  $E = 4$  episódios com *seeds* fixas reduz a variância estocástica da avaliação, estabilizando o sinal de seleção entre gerações.

Mutação Gaussiana *Element-wise*: Em vez de perturbar a totalidade dos  $|\theta|$  pesos em simultâneo — o que degradaria irrecuperavelmente a diversidade populacional em redes de grande dimensão —, implementa-se uma mutação esparsa controlada por uma máscara binária  $\mathbf{m} \in \{0, 1\}^{|\theta|}$ :

$$\theta_j^{(\text{filho})} = \theta_j^{(\text{pai})} + m_j \cdot \varepsilon_j, \quad m_j \sim \text{Bernoulli}(p_{mut}), \quad \varepsilon_j \sim \mathcal{N}(0, \sigma^2) \quad (2.2)$$

com  $p_{mut} = 0.1$ . Apenas 10% dos pesos são perturbados por operação, preservando a estrutura do controlador pai enquanto introduz variações localizadas. Esta formulação é equivalente ao *Evolution Strategies* de Salimans et al. (2017) com máscara esparsa.

*Sigma Decay* — Anilamento Adaptativo da Exploração: O desvio padrão  $\sigma$  controla a magnitude das perturbações. Para implementar a transição gradual de exploração agressiva para refinamento local,  $\sigma$  decai geometricamente a cada geração  $g$ :

$$\sigma_{g+1} = \max(\sigma_{\min}, \sigma_g \cdot \rho) \quad (2.3)$$

com  $\sigma_0 = 0.1$ ,  $\rho = 0.995$  e  $\sigma_{\min} = 0.01$ . Este decaimento assegura que nas fases iniciais o algoritmo explora amplamente a superfície  $J(\theta)$ , convergindo progressivamente para refinamento local do melhor ótimo encontrado.

Elitismo e Seleção por Ordenação: Após avaliar todos os  $K$  genomas, estes são ordenados por  $J(\theta_k)$  em ordem decrescente. Os  $e = \max(3, \lfloor 0.2 \cdot K \rfloor)$  melhores genomas (elites) são preservados intactos na nova geração, garantindo que o melhor comportamento descoberto nunca se perde por mutações desfavoráveis. Os restantes  $K - e$  genomas são gerados como mutações de cópias aleatórias dos elites. Para  $K = 30$ , preservam-se  $e = 6$  genomas elite por geração.

Otimização por Enxame de Partículas (PSO): O PSO modela cada controlador candidato como uma “partícula” que navega num espaço de busca multidimensional. A inovação crucial deste método reside na partilha de informação social: cada partícula ajusta a sua velocidade e posição baseada na sua melhor experiência pessoal (*pBest*) e na melhor experiência coletiva descoberta por todo o enxame (*gBest*). Estudos de Hassen e Amin (2025) demonstram que variantes modernas de PSO são extremamente eficientes na navegação reativa, embora a convergência do enxame indique tipicamente a descoberta de

um controlador estático que pode falhar em cenários onde a topologia do problema muda subitamente [8].

## 2.5. Abordagem 2: Multi-Agent Reinforcement Learning (MARL)

O MARL representa a fronteira da Aprendizagem Automática aplicada a sistemas coletivos, focando-se na aprendizagem de políticas adaptativas ( $\pi$ ) através da interação sequencial.

### 2.5.1. Processos de Decisão de Markov (MDPs) e Dec-POMDP

Fundamentos do Processo de Decisão de Markov (MDP): A base matemática da aprendizagem por reforço é o **Processo de Decisão de Markov (MDP)**, definido pela tupla  $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ :

- $\mathcal{S}$ : espaço de estados do ambiente (contínuo em robótica);
- $\mathcal{A}$ : espaço de ações ( $\mathbf{a}_i \in [-1, 1]^3$  nesta tese — deslocamento contínuo nos três eixos egocêntricos do robô: Frontal, Direito e Superior, escalado por um passo máximo de 0.2 m);
- $\mathcal{P}(s' | s, a)$ : dinâmica de transição — probabilidade de atingir  $s'$  ao executar  $a$  em  $s$ ;
- $\mathcal{R}(s, a, s') \in \mathbb{R}$ : sinal de recompensa escalar;
- $\gamma \in [0, 1)$ : fator de desconto temporal ( $\gamma = 0.99$  nesta implementação).

Funções de Valor e Equação de Bellman: O objetivo do agente é encontrar uma política  $\pi^*(a | s)$  que maximize o **retorno esperado descontado**  $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$ . A qualidade de uma política é quantificada por duas funções fundamentais.

A **Função de Valor de Estado**  $V^\pi(s)$  mede o retorno esperado ao seguir  $\pi$  a partir de  $s$ :

$$V^\pi(s) = \mathbb{E}_\pi \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right] \quad (2.4)$$

A **Função  $Q$**  (ou Função de Valor de Ação)  $Q^\pi(s, a)$  mede o retorno esperado ao executar  $a$  em  $s$  e depois seguir  $\pi$ :

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right] \quad (2.5)$$

Estas funções satisfazem a **Equação de Bellman**, que permite a sua estimação recursiva:

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s'} \mathcal{P}(s' | s, a) [\mathcal{R}(s, a, s') + \gamma V^\pi(s')] \quad (2.6)$$

A política ótima verifica a **Equação de Bellman de Optimalidade**:

$$Q^*(s, a) = \mathbb{E}_{s'} \left[ \mathcal{R}(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right] \quad (2.7)$$

Extensão para Dec-POMDP em Enxames: Em sistemas multiagente, o estado global  $s \in \mathcal{S}$  é inacessível a qualquer robô individual. O problema estende-se para o **Processo de Decisão de Markov Parcialmente Observável Descentralizado (Dec-POMDP)**,

definido pela tupla  $\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i \in \mathcal{N}}, \mathcal{P}, \mathcal{R}, \Omega, \{\mathcal{O}_i\}_{i \in \mathcal{N}}, \gamma \rangle$ , onde  $\mathcal{N} = \{1, \dots, N\}$  é o conjunto de agentes,  $\Omega$  é o espaço de observações conjuntas, e  $\mathcal{O}_i(s) \in \mathbb{R}^{111}$  é a observação local e parcial do agente  $i$  [11], [12], [19]. Cada robô mantém uma política descentralizada  $\pi_i(a_i | o_i)$  e o objetivo coletivo é maximizar o retorno conjunto:

$$J(\pi_1, \dots, \pi_N) = \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t \mathcal{R}(s_t, \mathbf{a}_t) \right], \quad \mathbf{a}_t = (a_1^t, \dots, a_N^t) \quad (2.8)$$

### 2.5.2. Otimização de Políticas: Do Actor-Critic ao PPO e SAC

Arquitetura Actor-Critic: Tanto o PPO como o SAC assentam na arquitetura **Actor-Critic**, onde duas redes são treinadas em conjunto:

- **Ator** ( $\pi_\theta$ ): mapeia a observação  $o_i$  para uma distribuição sobre ações. Em execução descentralizada, **apenas o ator é necessário** a bordo de cada robô.
- **Crítico** ( $V_\phi$  ou  $Q_\psi$ ): rede auxiliar que estima o retorno esperado, utilizada **exclusivamente durante o treino** para calcular gradientes de atualização do ator.

PPO — *Clipped Surrogate Objective*: O PPO (*Proximal Policy Optimization*) [20] é um algoritmo *on-policy*: recolhe *rollouts* com a política atual e atualiza os parâmetros através de um objetivo *clipped* que previne passos de gradiente destabilizadores. Seja a **razão de importância**:

$$r_t(\theta) = \frac{\pi_\theta(a_t | o_t)}{\pi_{\theta_{\text{old}}}(a_t | o_t)} \quad (2.9)$$

O objetivo PPO *clipped* é:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[ \min \left( r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon) \hat{A}_t \right) \right] \quad (2.10)$$

com  $\varepsilon = 0.2$ . A estimativa da vantagem  $\hat{A}_t$  é calculada via **Estimação de Vantagem Generalizada (GAE)** [21]:

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \quad (2.11)$$

O objetivo final inclui ainda um erro de regressão do crítico e um bônus de entropia:

$$L^{\text{PPO}}(\theta, \phi) = L^{\text{CLIP}}(\theta) - c_1 \mathbb{E}_t [(V_\phi(s_t) - V_t^{\text{targ}})^2] + c_2 \mathbb{E}_t [H(\pi_\theta(\cdot | o_t))] \quad (2.12)$$

SAC — Maximização de Entropia: O SAC (*Soft Actor-Critic*) [22] é um algoritmo *off-policy* que aumenta o objetivo de RL com o termo de entropia de Shannon  $H(\pi) = -\mathbb{E}[\log \pi(a | s)]$ , promovendo a **exploração máxima** consistente com o desempenho na tarefa:

$$J(\pi) = \mathbb{E}_{\rho_\pi} \left[ \sum_{t=0}^{\infty} \gamma^t \left( r_t + \alpha H(\pi(\cdot | s_t)) \right) \right] \quad (2.13)$$

onde  $\alpha > 0$  é o **coeficiente de temperatura**, que controla o equilíbrio entre exploração e recompensa ( $\alpha = 0.1$  nesta implementação). A **Função de Valor Suave** incorpora

diretamente o termo de entropia:

$$V^*(s) = \mathbb{E}_{a \sim \pi} [Q^*(s, a) - \alpha \log \pi(a \mid s)] \quad (2.14)$$

O ator é atualizado minimizando a divergência KL relativamente à distribuição de Boltzmann  $\propto \exp(Q/\alpha)$ :

$$J_\pi(\phi) = \mathbb{E}_{s_t \sim \mathcal{D}} [\mathbb{E}_{a_t \sim \pi_\phi} [\alpha \log \pi_\phi(a_t \mid o_t) - Q_\psi(s_t, a_t)]] \quad (2.15)$$

O SAC mantém dois críticos  $Q_{\psi_1}, Q_{\psi_2}$  cujo mínimo é usado no alvo (*double Q-trick*), reduzindo o viés de sobre-estimação. As transições  $(s_t, a_t, r_t, s_{t+1})$  são armazenadas num **Replay Buffer** de capacidade  $|\mathcal{D}| = 500\,000$ , permitindo a reutilização *off-policy* de experiências passadas.

O Desafio da Não-Estacionaridade: A principal barreira teórica na aplicação de MARL a enxames é a não-estacionaridade do ambiente. Num sistema multiagente, as transições de estado para um agente  $i$  dependem das ações de todos os outros agentes. Como todos os robôs estão a aprender e a alterar as suas políticas simultaneamente, a dinâmica do ambiente é instável, o que viola a propriedade fundamental de Markov e dificulta a convergência dos algoritmos tradicionais de agente único.

Arquitetura CTDE e Escalabilidade com GNNs: Para mitigar a instabilidade, utiliza-se a arquitetura de **Treinamento Centralizado com Execução Descentralizada** (CTDE) [23]. Nesta abordagem, um componente “Crítico” tem acesso ao estado global durante a fase de treino para estabilizar os sinais de recompensa, enquanto o “Ator” (controlador do robô) utiliza apenas observações locais durante a fase de execução [16].

Contudo, para garantir que o enxame é verdadeiramente escalável, a investigação moderna tem integrado **Graph Neural Networks (GNNs)** e mecanismos de atenção espacial.

### 2.5.3. Fundamentos de Graph Neural Networks e Mecanismos de Atenção

Representação em Grafo do Enxame: O enxame de  $N$  robôs é modelado como um grafo dinâmico  $G_t = (V_t, E_t)$ , onde  $V_t = \{1, \dots, N\}$  é o conjunto de nós (robôs). Em vez de um corte rígido por raio de comunicação, adota-se um **grafo completamente ligado** ( $E_t = \{(i, j) : i \neq j\}$ ) cujas arestas são ponderadas dinamicamente pelos coeficientes de atenção  $\alpha_{ij}$  (Equação 2.19). A topologia efetiva é assim aprendida e reflete, a cada passo, a relevância relativa de cada vizinho para a decisão do agente — arestas irrelevantes recebem peso próximo de zero, emulando o efeito de um corte de proximidade sem o impor *a priori*.

Propagação de Mensagens (*Message Passing*): O paradigma geral das GNNs é o framework MPNN (*Message Passing Neural Network*). A representação  $\mathbf{h}_i^{(l)}$  do nó  $i$  na camada  $l$  é atualizada por:

$$\mathbf{h}_i^{(l+1)} = \text{UPDATE}\left(\mathbf{h}_i^{(l)}, \text{AGG}\left(\left\{\mathbf{m}_{j \rightarrow i}^{(l)} : j \in \mathcal{N}(i)\right\}\right)\right) \quad (2.16)$$

onde  $\mathbf{m}_{j \rightarrow i}^{(l)} = \text{MSG}(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)})$  é a mensagem do vizinho  $j$ , e  $\text{AGG}(\cdot)$  é uma função invariante à permutação (soma, média ou máximo). Esta invariância é fundamental: a operação é independente da ordem dos vizinhos, garantindo que o enxame é tratado como um conjunto não-ordenado.

Mecanismo de Atenção Escalada: A arquitetura implementada (GNNAgent3D) usa o mecanismo de **atenção de produto interno escalado** [24], distinto da formulação aditiva original das *Graph Attention Networks* [25]. Para o agente  $i$ , codificam-se as suas características locais  $\mathbf{h}_i^{env} = f_{enc}(\mathbf{o}_i^{env}) \in \mathbb{R}^d$  e as dos  $N-1$  vizinhos  $\mathbf{h}_j^{nbr} = f_{nbr}(\mathbf{n}_{j,i}) \in \mathbb{R}^d$ , formando a matriz de contexto:

$$\mathbf{H} = [\mathbf{h}_i^{env}; \mathbf{h}_1^{nbr}; \dots; \mathbf{h}_{N-1}^{nbr}] \in \mathbb{R}^{N \times d} \quad (2.17)$$

As projeções de consulta, chave e valor são calculadas pelas matrizes treináveis  $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in \mathbb{R}^{d \times d}$ :

$$\mathbf{Q} = \mathbf{h}_i^{env} \mathbf{W}^Q \in \mathbb{R}^{1 \times d}, \quad \mathbf{K} = \mathbf{H} \mathbf{W}^K \in \mathbb{R}^{N \times d}, \quad \mathbf{V} = \mathbf{H} \mathbf{W}^V \in \mathbb{R}^{N \times d} \quad (2.18)$$

Os **coeficientes de atenção**  $\alpha_{ij}$  medem a importância que o agente  $i$  atribui ao vizinho  $j$ :

$$\alpha_i = \text{softmax}\left(\frac{\mathbf{Q} \mathbf{K}^\top}{\sqrt{d}}\right) \in \mathbb{R}^{1 \times N} \quad (2.19)$$

A divisão por  $\sqrt{d}$  previne a saturação do *softmax* em espaços de alta dimensão. O **pool de mensagens agregado** é:

$$\mathbf{m}_i = \alpha_i \mathbf{V} = \sum_{j=0}^{N-1} \alpha_{ij} \mathbf{h}_j \in \mathbb{R}^d \quad (2.20)$$

A representação final concatena o vetor ego com o pool de mensagens e gera a ação contínua:

$$\mathbf{z}_i = [\mathbf{h}_i^{env}; \mathbf{m}_i] \in \mathbb{R}^{2d}, \quad \mathbf{a}_i = f_{actor}(\mathbf{z}_i) \in [-1, 1]^3 \quad (2.21)$$

Ao modelar o enxame como um grafo dinâmico  $G = (V, E)$ , onde os nós são robôs e as arestas são ligações de comunicação, é possível aprender políticas invariantes ao número de agentes. Como demonstrado por Chen et al. (2024), esta abordagem permite o **Zero-Shot Transfer**, onde um controlador treinado com um pequeno grupo de agentes pode ser diretamente aplicado a enxames de centenas de robôs sem necessidade de retreino [16]. Esta tese visa precisamente validar se este paradigma de aprendizagem adaptativa supera as limitações de rigidez das abordagens bio-inspiradas em cenários de stress dinâmico [6].

## CAPÍTULO 3

### Estado da Arte

#### 3.1. Introdução ao Capítulo

Este capítulo apresenta uma Revisão Sistemática da Literatura (SLR) rigorosa, centrada nos paradigmas de controlo de enxames discutidos no capítulo anterior. Numa área em constante evolução como a Robótica de Enxame, onde as publicações científicas transitaram das conceptualizações iniciais da década de 1990 para aplicações críticas e complexas em 2025, torna-se imperativo adotar um método de análise que seja simultaneamente metódico e reproduzível. O objetivo primordial desta revisão não é apenas sumarizar o conhecimento existente, mas sim analisar de que forma a Aprendizagem Automática — especificamente o *Multi-Agent Reinforcement Learning* (MARL) — e os métodos de Otimização Bio-inspirada (como o PSO e AE) têm convergido ou divergido na resolução de problemas de coordenação descentralizada [5].

A motivação para esta análise sistemática reside na observação de que a comunidade científica tem, historicamente, operado em silos: a literatura de MARL foca-se predominantemente na estabilidade do treino sob condições de não-estacionaridade, enquanto a literatura de otimização prioriza a robustez de controladores estáticos. Esta divisão tem dificultado a realização de confrontos diretos que validem qual o paradigma mais resiliente perante o chamado “Deployment Gap” (fosso de implantação). Conforme discutido por Majid et al. (2024), a falta de um referencial comum de desempenho impede uma compreensão clara de quando a adaptabilidade *online* compensa a complexidade computacional do treino em larga escala [7].

Desta forma, a estrutura desta SLR foca-se em identificar e mapear as lacunas de *benchmarking* que justificam a necessidade desta dissertação. Através da taxonomia aqui apresentada, serão exploradas as tendências de escalabilidade via *Graph Neural Networks* (GNNs), defendidas por Chen et al. (2024), e as abordagens de hibridização emergentes que tentam fundir a eficácia dos algoritmos evolutivos com a flexibilidade do RL [16], [26]. Ao finalizar este capítulo, será possível sustentar a validade da metodologia proposta no Capítulo 4 como a resposta necessária às limitações encontradas na literatura contemporânea [15].

#### 3.2. Metodologia da Revisão Sistemática

Para garantir o rigor científico, a reprodutibilidade e a transparência no processo de seleção da informação, foi adotada uma metodologia baseada no protocolo PRISMA (*Preferred Reporting Items for Systematic Reviews and Meta-Analyses*, versão 2020). A adoção deste

protocolo permite mitigar o viés de seleção e assegurar que a análise crítica das tecnologias MARL e de otimização bio-inspirada se fundamenta numa amostra representativa e qualificada da literatura contemporânea.

### 3.2.1. Critérios de Seleção e Rigor Técnico

A pesquisa priorizou bases de dados com elevado fator de impacto na área da computação e robótica: **IEEE Xplore**, **ACM Digital Library** e **Scopus**. Fontes genéricas ou não indexadas foram descartadas para garantir a qualidade da *baseline*.

O critério de inclusão principal focou-se na formalização matemática do problema: foram selecionados apenas estudos que modelassem o controlo de enxame como um **Processo de Decisão de Markov Parcialmente Observável Descentralizado (Dec-POMDP)** ou problemas de otimização equivalentes. Artigos anteriores a 2020 ou que apresentassem abordagens puramente teóricas sem validação em simulação dinâmica foram excluídos na fase de triagem, garantindo que o Estado da Arte reflete apenas as tecnologias de ponta atuais.

### 3.2.2. Protocolo de Pesquisa e Bases de Dados

A fase de levantamento bibliográfico foi conduzida entre Setembro e Outubro de 2025, recorrendo a fontes de dados fidedignas e amplamente reconhecidas pela comunidade científica. O fluxo de trabalho, desde a captura inicial até à leitura técnica detalhada, foi centralizado na plataforma **Mendeley**. Esta ferramenta foi crucial não apenas para a organização dos artigos, mas também para a gestão automática de metadados, deteção de duplicados e anotação técnica, garantindo a integridade e a rastreabilidade da bibliografia apresentada ao longo da dissertação.

As bases de dados científicas indexadas foram selecionadas estrategicamente para cobrir tanto a especificidade técnica da engenharia como o impacto global das publicações:

- **IEEE Xplore e ACM Digital Library:** Consultadas pela sua especialização em computação, robótica e engenharia eletrotécnica, onde se encontram os principais desenvolvimentos em algoritmos de controlo descentralizado.
- **Scopus e Google Scholar:** Utilizadas para garantir uma abrangência multidisciplinar e capturar artigos de alto impacto e revisões de literatura que estabelecem o estado da arte em inteligência artificial.

A *string* de pesquisa foi desenhada utilizando operadores booleanos para capturar a interseção precisa entre o domínio da Robótica de Enxame e as soluções tecnológicas em análise:

(*"Swarm Robotics"OR "Multi-Robot Systems"*) AND (*"Reinforcement Learning"OR "MARL"OR "Bio-inspired Optimization"OR "PSO"*) AND (*"Control"OR "Navigation"*).

Esta combinação de termos permitiu filtrar estudos que abordassem simultaneamente o paradigma de controlo (MARL ou Otimização) e a aplicação prática em enxames (Navegação/Controlo), excluindo automaticamente literatura focada em agentes isolados ou aplicações puramente teóricas sem componente robótica.

### 3.2.3. Processo de Seleção (Fluxo PRISMA)

O rigor do processo de seleção é fundamental para assegurar que o corpo de literatura analisado é robusto e diretamente relevante para os objetivos desta dissertação. O fluxo de trabalho seguiu três fases críticas, documentadas detalhadamente no fluxograma da Figura 3.1:

- (1) **Identificação:** A pesquisa inicial nas bases de dados selecionadas retornou um total de **145 artigos**. Todos os registos foram importados para o software **Mendeley**, facilitando a triagem inicial e a deteção de duplicados. Após a remoção automática e manual de 25 duplicados, restaram 120 registos únicos para análise de mérito.
- (2) **Triagem (Screening):** Foram **excluídos 60 artigos após a leitura integral de títulos e resumos**. O principal critério de exclusão nesta fase residiu no desvio do âmbito técnico, nomeadamente estudos focados exclusivamente em biologia animal teórica ou sistemas robóticos de agente único, que não oferecem contributos para a problemática da coordenação descentralizada.
- (3) **Elegibilidade:** Dos restantes 60 artigos lidos na íntegra, 34 foram excluídos por apresentarem falta de dados comparativos quantitativos ou metodologias obsoletas que não consideravam ambientes dinâmicos e ruidosos. Esta fase foi crucial para garantir que apenas estudos com validade estatística e relevância para o controlo de enxames moderno fossem retidos.

No final deste processo, **26 artigos** foram selecionados para a fase de análise qualitativa e extração de dados técnicos, servindo de base para a taxonomia apresentada de seguida.

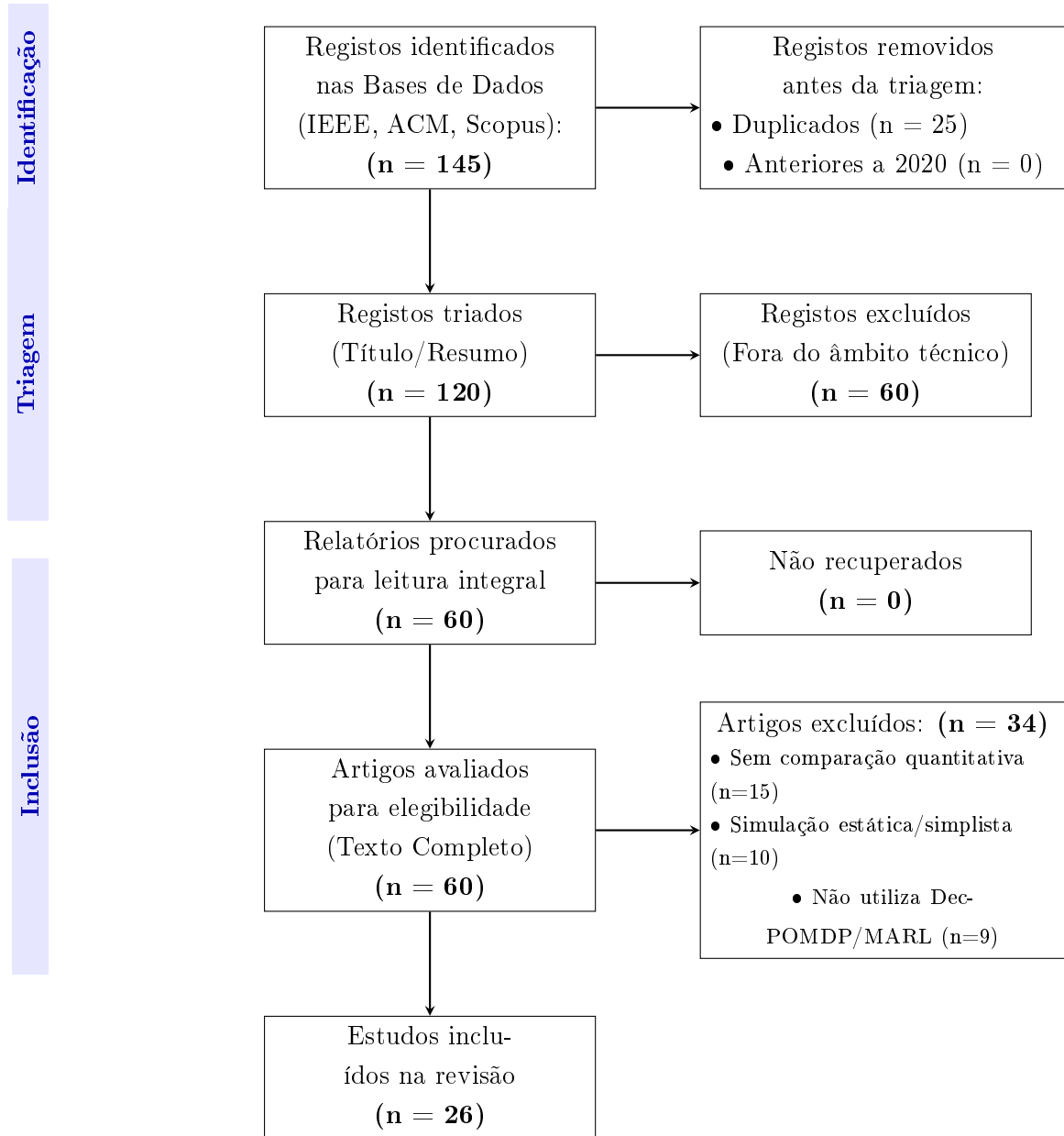


FIGURA 3.1. Fluxograma PRISMA 2020: Seleção focada em estudos recentes e quantitativos.

### 3.3. Análise dos Melhores Resultados

#### 3.3.1. Subgrupo A: Otimização Bio-inspirada (A Referência de Eficiência)

Os estudos mais relevantes neste grupo, como o de **Hassen e Amin (2025)**, demonstram que variantes híbridas de PSO continuam a deter o recorde de eficiência energética em mapas estáticos. Os resultados indicam que, quando o ambiente não muda, a otimização *offline* converge para soluções com 98% de otimalidade, servindo como uma *baseline* extremamente difícil de bater para o RL. Contudo, estes mesmos estudos reportam quebras de performance superiores a 40% quando são introduzidos obstáculos móveis não previstos no treino inicial [8].

### 3.3.2. Subgrupo B: MARL e Escalabilidade (O Estado da Arte)

A literatura de MARL abandonou abordagens simplistas (como Q-Learning tabular) em favor de arquiteturas baseadas em grafos. O trabalho de **Chen et al. (2024)** demonstra que a modelação do sistema como um grafo, processado por **redes neuronais de grafos (GNNs)** com atenção, reduz o erro de estimação face a métodos de agregação padrão e, decisivamente, preserva a transferibilidade da política quando o número de agentes varia (*Zero-Shot Transfer*) — validando o uso de GNNs para mitigar o problema da dimensionalidade em sistemas multiagente descentralizados [16].

Recentemente, Xiao et al. (2026) propuseram o método CSPER, destacando que abordagens de RL sofrem de fraca exploração em ambientes complexos de navegação devido a recompensas esparsas [27]. Este trabalho reforça a necessidade de mecanismos de recompensa densos, como os propostos nesta tese. Paralelamente, Roveda (2026) explorou a hibridização entre RL e Algoritmos Evolutivos para manipulação adaptativa, sugerindo que a combinação dos dois mundos pode mitigar a incerteza em ambientes não estruturados [28].

Mais próximo da formulação adotada nesta dissertação, Yigit et al. (2026) propõem um sistema de navegação de enxames totalmente descentralizado, modelado como um *Multi-Agent Markov Decision Process* (MAMDP) e treinado exclusivamente com **PPO** e *parameter sharing*, sem comunicação explícita entre agentes [29]. Os autores reportam convergência rápida e taxas de sucesso quase perfeitas numa arena 2D com obstáculos circulares estáticos, recorrendo a uma função de recompensa multicomponente (progresso, penalização de colisões, *team bonus*) muito semelhante à aqui empregue, o que valida empiricamente o paradigma PPO descentralizado adotado como *baseline*. Contudo, este trabalho evidencia precisamente as lacunas que a presente tese pretende preencher: o espaço de observação é de dimensão fixa (8), impossibilitando o *Zero-Shot Transfer* para enxames de tamanho variável; a avaliação restringe-se à comparação contra uma política aleatória, sem testes estatísticos nem cenários de *stress*; e os próprios autores remetem para *future work* a inclusão de obstáculos dinâmicos, agentes heterogêneos e falhas de comunicação — aspetos já contemplados na metodologia desta dissertação através de *Graph Neural Networks*, cenários de labirinto com campo geodésico e injeção controlada de falhas de agentes.

### 3.4. O Desafio do *Deployment Gap* em Robótica de Enxame

Uma política de controlo que convirja para taxas de sucesso próximas de 100% em simulação não oferece, por si só, qualquer garantia de desempenho num robô físico. O fosso entre os dois domínios — designado *reality gap* ou *Deployment Gap* — é apontado por Kegeleirs et al. (2025) como o principal entrave à transição da robótica de enxame do laboratório para aplicações reais [15]. A causa é estrutural: o treino por reforço otimiza a política para a distribuição exata de transições  $\mathcal{P}(s' | s, a)$  do simulador, pelo que qualquer discrepância sistemática entre essa distribuição e a do mundo real é explorada como se

fosse uma regularidade legítima do ambiente, conduzindo a comportamentos que falham — por vezes catastroficamente — na plataforma física.

### 3.4.1. Fontes do Fosso de Realidade

As discrepâncias que alimentam o *Deployment Gap* podem agrupar-se em três classes:

- **Dinâmica de atuação e física não modelada:** um simulador cinemático — como o adotado nesta dissertação, em que a ação é um deslocamento egocêntrico — ignora a inércia, a aceleração finita dos motores, a fricção assimétrica entre rodas, a folga mecânica e o escorregamento. Num chassis *differential-drive* real (e-puck, Khepera IV), um comando de velocidade não se traduz instantaneamente na deslocação pretendida, e pequenos enviesamentos de calibração entre motores acumulam-se em desvios de trajetória que a política treinada nunca observou.
- **Ruído e enviesamento sensorial:** a observação simulada (bússola de *homing*, LiDAR, posição relativa de vizinhos) é exata ou corrompida apenas por ruído idealizado. Os sensores reais introduzem ruído não-gaussiano, latência, ângulos mortos, reflexões espúrias no LiDAR e erros de odometria que derivam ao longo do tempo. Uma política que dependa de leituras precisas da direção ou da distância degrada-se à medida que estas se tornam ruidosas.
- **Comunicação e coordenação:** o sinal de recrutamento estigmérgico entre vizinhos é, no simulador, transmitido de forma instantânea e fiável. Em hardware real, a comunicação sofre de latência, perda de pacotes, alcance limitado e colisões de mensagens — e a percepção da posição dos vizinhos depende ela própria de sensores imperfeitos.

### 3.4.2. Agravamento à Escala do Enxame

O *Deployment Gap* é particularmente severo em sistemas multiagente. Ao contrário de um único robô, num enxame os pequenos erros de cada agente **compõem-se e interagem**: um desvio de trajetória individual altera a configuração espacial percebida pelos vizinhos, que ajustam o seu comportamento, propagando a perturbação pelo coletivo. Comportamentos coordenados que emergem de um equilíbrio delicado em simulação (a sincronização de três agentes na porta cooperativa, o cerco a 360 do alvo móvel) são precisamente os mais frágeis perante este efeito de amplificação, pois dependem de uma coordenação fina que o ruído real tende a dissolver.

### 3.4.3. Estratégias de Mitigação

A literatura propõe várias famílias de técnicas para estreitar o fosso. A **aleatorização de domínio** (*domain randomization*) é a mais difundida: durante o treino, randomizam-se sistematicamente os parâmetros físicos e sensoriais do simulador (massas, fricções, ganhos de motor, níveis de ruído), forçando a política a ser robusta a uma *distribuição* de dinâmicas em vez de a uma única, de modo que o mundo real surja como apenas mais uma amostra dessa distribuição [30]. Complementarmente, a **identificação de sistema**

(*system identification*) calibra os parâmetros do simulador a partir de medições reais, reduzindo o enviesamento à partida; e o **ajuste fino** (*fine-tuning*) no robô físico, ou em simuladores de fidelidade crescente, adapta a política às condições efetivas de operação.

### 3.4.4. Posicionamento desta Dissertação

Embora a validação *sim-to-real* permaneça fora do âmbito experimental deste trabalho (Secção 7.2), várias opções de desenho foram tomadas no sentido de *antecipar* o fosso, e não de o ignorar. A observação é mantida estritamente **local e parcial** (LiDAR limitado a 5m, sem acesso ao estado global), replicando as restrições sensoriais de plataformas como o e-puck; introduz-se **observabilidade parcial genuína** ao nível dos obstáculos (detetáveis apenas localmente); a robustez é explicitamente testada através da **injeção de falhas** de 10% dos agentes a meio do episódio (Secção 6.11); e o ambiente inclui **perturbações dinâmicas** (obstáculos e alvo móveis). A elevada resiliência observada perante falhas sugere que a partilha de parâmetros confere ao enxame uma redundância funcional favorável à transferência. A validação empírica em hardware, recorrendo às técnicas de *domain randomization* e *system identification* acima descritas, constitui a extensão natural deste trabalho (Secção 7.4).

### 3.5. Lacunas Identificadas e Oportunidade de Melhoria

A revisão sistemática da literatura permitiu consolidar a evidência de que a principal barreira ao avanço da robótica de enxame não reside na escassez de algoritmos, mas sim na **falha sistemática de rigor comparativo** entre paradigmas distintos. Observou-se que as comunidades de *Multi-Agent Reinforcement Learning* (MARL) e de Otimização Bio-inspirada operam em silos metodológicos quase estanques:

- A comunidade de aprendizagem foca-se primordialmente na estabilidade do treino e na superação da não-estacionaridade, testando novos algoritmos (ex: MAPPO, QMIX) contra variantes de RL, negligenciando baselines de otimização consolidada [13].
- A comunidade de otimização foca-se na eficiência energética e otimalidade em cenários estáticos, ignorando a flexibilidade que a aprendizagem adaptativa pode oferecer em tempo real [7].

Esta fragmentação resulta numa negligência crítica de testes de *stress* dinâmico, como a falha súbita de 10% do enxame ou alterações abruptas na topologia de comunicação. Conforme identificado por Kegeleirs et al. (2025), esta ausência de testes em ambientes ruidosos e imprevisíveis constitui o principal entrave à transição dos enxames para aplicações industriais e de busca e salvamento [15].

Esta tese pretende melhorar este cenário ao propor um *benchmark* que transcende a simples métrica de performance final, focando-se na **capacidade de recuperação e adaptação** perante perturbações dinâmicas. A oportunidade de melhoria reside na utilização de *Graph Neural Networks* (GNNs), que permitem modelar o enxame como um grafo dinâmico  $G = (V, E)$  [16]. Esta abordagem garante a invariância à permutação e a

escalabilidade necessárias para superar a rigidez dos métodos bio-inspirados, permitindo que o sistema mantenha a coesão e a eficácia mesmo quando a estrutura do coletivo é alterada por eventos externos imprevistos.

### 3.6. Artigos-Chave e Ligação ao Problema

Para fundamentar cientificamente a definição do problema desta tese, foram selecionados seis artigos críticos que mapeiam os avanços e as limitações tecnológicas de 2020 a 2025:

- (1) **Chen et al. (2024)**: Este trabalho é fundamental por demonstrar a eficácia das **GNNs** na minimização do erro de estimação em enxames totalmente descentralizados, validando a arquitetura tecnológica de base escolhida para o simulador desta tese [16].
- (2) **Elrod et al. (2025)**: Explora o potencial de arquiteturas baseadas em **Transformers** para a coordenação multiagente, destacando como a atenção espacial é superior aos métodos de agregação simples para manter a coesão do grupo em tarefas dinâmicas [31].
- (3) **Heimann et al. (2024)**: Introduz o conceito de **Safe MARL**, integrando métodos de verificação em tempo real no processo de aprendizagem para mitigar o risco de colisões catastróficas, uma preocupação central para a implantação em sistemas físicos reais [18].
- (4) **Iskandar et al. (2023)**: Aplica aprendizagem por reforço profundo à navegação de um enxame de robôs em simulação, representando a linha de investigação de RL aplicado a *swarm* que esta dissertação estende com uma **comparação sistemática** face aos métodos bio-inspirados, ainda rara na literatura [32].
- (5) **Kegeleirs et al. (2025)**: Identifica criticamente as limitações na transição da simulação para a realidade (*sim-to-real*), apontando a ausência de testes em ambientes ruidosos e com falhas de comunicação como o principal obstáculo à adoção industrial [15].
- (6) **Wang et al. (2025)**: Propõe o *framework* **LNS2+RL**, demonstrando que a hibridização entre pesquisa local e aprendizagem por reforço é a via mais eficaz para resolver problemas de *pathfinding* em larga escala com obstáculos móveis [26].

### 3.7. Conclusão Parcial

O Estado da Arte validou a escolha da aprendizagem automática avançada, particularmente através de modelos de MARL descentralizados, como o paradigma com maior potencial para o desenvolvimento de sistemas de controlo verdadeiramente adaptativos [6]. A literatura recente confirma que, embora os métodos bio-inspirados continuem a definir linhas de base robustas para tarefas estáticas, a sua limitação de adaptação *online* restringe a sua utilidade em ambientes de alta incerteza [6].

No entanto, a manifesta escassez de dados comparativos quantitativos entre as versões modernas de MARL (utilizando GNNs e Transformers) e variantes avançadas de

PSO e Algoritmos Evolutivos em tarefas dinâmicas justifica a necessidade da metodologia proposta no capítulo seguinte [7]. Este trabalho propõe-se a preencher este vazio, fornecendo as métricas necessárias para determinar quando a complexidade computacional da aprendizagem compensa, de facto, a robustez pré-programada da otimização tradicional.

### 3.8. Síntese Comparativa da Literatura

A Tabela 3.1 apresenta uma síntese dos estudos mais relevantes analisados, destacando a abordagem utilizada e as limitações que motivam a presente dissertação.

TABELA 3.1. Resumo dos artigos mais relevantes e identificação de lacunas.

| Referência                   | Paradigma           | Contributo Principal                                                                                                | Limitação / Gap Identificada                                                                                                |
|------------------------------|---------------------|---------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Chen et al. (2024) [16]      | MARL (GNN)          | Demonstra transferibilidade (Zero-Shot) a dimensões de enxame não vistas usando GNNs.                               | Foca-se na minimização do erro de estimação, sem obstáculos físicos complexos.                                              |
| Hassen & Amin (2025) [8]     | Bio-inspirado (PSO) | Alta eficiência energética (98% otimalidade) em mapas estáticos.                                                    | Performance degrada-se 40% com obstáculos móveis (falta de adaptação online).                                               |
| Heimann et al. (2024) [18]   | Safe MARL           | Introduz verificação em tempo real para evitar colisões durante o treino.                                           | Avaliado em ambientes simplificados, sem comparação com métodos evolutivos.                                                 |
| Iskandar et al. (2023) [32]  | MARL (Deep RL)      | Aplica aprendizagem por reforço profundo à navegação de um enxame de robôs em simulação.                            | Não compara com métodos bio-inspirados; cenário de navegação simples, sem falhas de agentes.                                |
| Kegeleirs et al. (2025) [15] | Review              | Identifica o "Deployment Gap" como barreira principal.                                                              | Trabalho teórico; não propõe uma solução algorítmica direta.                                                                |
| Yigit et al. (2026) [29]     | MARL (PPO)          | PPO descentralizado com <i>parameter sharing</i> e recompensa multicomponente; convergência rápida em navegação 2D. | Observação de dimensão fixa (sem Zero-Shot); só compara com política aleatória; obstáculos estáticos e sem testes de falha. |

Esta análise tabular reforça que, embora existam soluções robustas para problemas isolados (escalabilidade ou segurança), falta uma validação integrada que compare a **adaptabilidade dinâmica** do MARL contra a **eficiência estática** do PSO sob condições de falha.

[ Página intencionalmente deixada em branco. ]

## Desenvolvimento do Ambiente de Simulação

### 4.1. Arquitetura de Software e Stack Tecnológica

Para assegurar a viabilidade técnica e a reprodutibilidade dos resultados, a implementação do sistema foi desenvolvida com base nas tecnologias padrão da indústria para Inteligência Artificial. A arquitetura adotada privilegia a modularidade, a flexibilidade e o desempenho computacional, permitindo a substituição independente de qualquer componente sem comprometer a integridade do *pipeline* experimental.

#### 4.1.1. Ambiente de Simulação e Interação

O desenvolvimento foi centralizado na linguagem **Python 3.x**, escolhida pela sua vasta comunidade científica e pelo ecossistema de bibliotecas especializadas em aprendizagem automática. A interface do simulador segue o estilo da **API *parallel* do PettingZoo** — um padrão aberto para ambientes de agentes múltiplos —, devolvendo a cada passo dicionários de observações, recompensas e sinais de terminação indexados por agente. Esta convenção abstrai a física interna e facilita a troca entre algoritmos sem reescrever o código base do ambiente [13].

A observação de cada agente é estritamente **local e parcial**, garantindo que o sistema opera num regime verdadeiramente descentralizado. Não existe qualquer componente com acesso ao estado global durante a fase de execução, replicando fielmente as limitações sensoriais de plataformas físicas como o Khepera IV ou o e-puck.

#### 4.1.2. Implementação dos Algoritmos

A construção das redes neuronais e o treino dos agentes MARL apoiaram-se em *frameworks* de computação tensorial de alto desempenho:

- **PyTorch:** Utilizado para a implementação da *Graph Neural Network* (GNN) do algoritmo evolutivo, devido ao seu suporte nativo para grafos dinâmicos e à facilidade de depuração de operações tensoriais personalizadas.
- **Stable-Baselines3:** Biblioteca de referência para a implementação dos algoritmos PPO e SAC, fornecendo implementações estáveis e bem validadas pela comunidade científica, com suporte a ambientes vetorizados paralelos.
- **Paralelização Multiprocesso (CPU):** A aceleração do treino assenta na execução paralela de múltiplos ambientes de simulação em processos independentes — `SubprocVecEnv` para o PPO e o SAC e `multiprocessing.Pool` para a avaliação da população evolutiva —, distribuindo a carga pelos vários núcleos de CPU dos servidores de cálculo. O simulador é implementado em Python/NumPy e

executa em CPU; as redes envolvidas (a MLP [256, 256] da *Stable-Baselines3* e a *GNNAgent3D* de  $\approx 8k$  parâmetros) são suficientemente compactas para não exigirem GPU dedicada, sendo o *throughput* de treino dominado pela simulação física do ambiente e não pela passagem pelas redes.

### 4.1.3. Arquitetura Geral do Simulador

A Figura 4.1 ilustra o fluxo de dados em ciclo fechado do simulador, onde cada agente iterativamente recolhe observações e executa ações sob a validação de um motor de física contínuo.

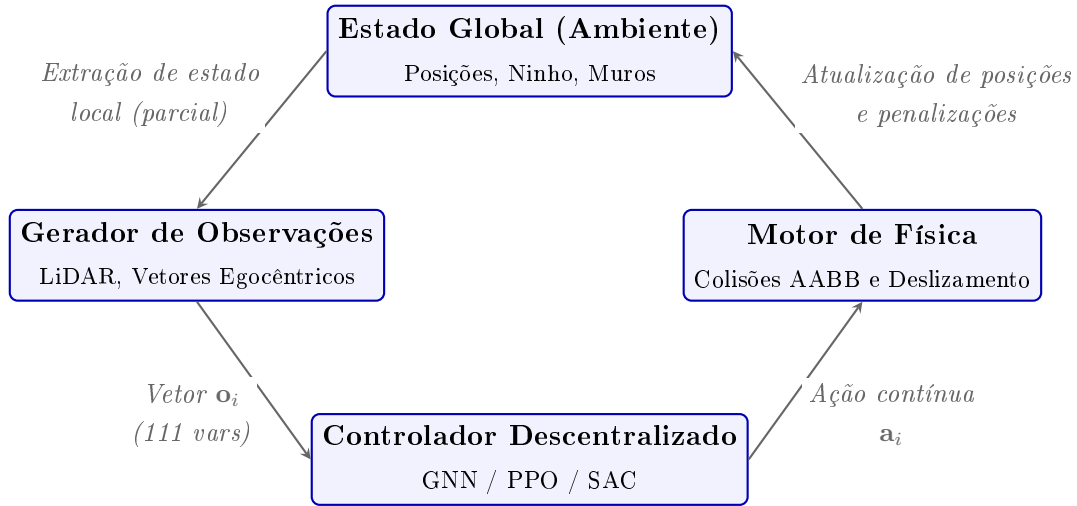


FIGURA 4.1. Diagrama da arquitetura do simulador e fluxo de interação agente-ambiente.

## 4.2. O Ambiente de Simulação

O simulador foi desenvolvido de raiz em **Python**, projetado para colmatar o *Deployment Gap* (fosso de implantação) através da modelação de alta fidelidade da física dos agentes, das restrições sensoriais e das perturbações dinâmicas do ambiente [15]. Todos os algoritmos são avaliados no mesmo ambiente, garantindo a paridade da comparação.

### 4.2.1. Modelo Físico e Hiperparâmetros

A arena de simulação consiste numa área circular de raio  $r_{arena} = 15\text{m}$ , onde  $N = 20$  agentes homogêneos operam em simultâneo. A Tabela 4.1 sintetiza os hiperparâmetros físicos e arquiteturais aplicados.

TABELA 4.1. Hiperparâmetros reais utilizados na simulação e no treino dos modelos.

| Categoria             | Hiperparâmetro                                 | Valor Real                        |
|-----------------------|------------------------------------------------|-----------------------------------|
| <b>Ambiente</b>       | Raio da Arena ( $r_{arena}$ )                  | 15.0 m                            |
|                       | Número de Agentes ( $N$ )                      | 20                                |
|                       | Número de Obstáculos Dinâmicos                 | 100                               |
|                       | Raio dos Obstáculos ( $r_{obs}$ )              | 0.2 m                             |
|                       | Velocidade dos Obstáculos ( $v_{obs}$ )        | 0.02 m/s                          |
|                       | Raio do Ninho ( $r_{nest}$ )                   | 1.5 m                             |
|                       | Velocidade do Ninho ( $v_{nest}$ )             | 0.015 m/s                         |
|                       | Limite do Temporizador de Fome ( $hunger$ )    | 250 passos (só Sandbox)           |
|                       | Cooperação Mínima para Alimentar ( $k_{min}$ ) | 3 agentes                         |
|                       | Fator de Progresso (PBRs)                      | 10.0                              |
|                       | Resolução do Campo Geodésico                   | 0.4 m                             |
| <b>PPO</b>            | Taxa de Aprendizagem ( $lr$ )                  | $10^{-4}$                         |
|                       | Passos por Rollout ( $n_{steps}$ )             | 64 (por CPU)                      |
|                       | Batch Size                                     | 512                               |
|                       | Número de Épocas ( $n_{epochs}$ )              | 4                                 |
|                       | Arquitetura da Rede                            | MLP [256, 256]                    |
|                       | Número de CPUs Paralelos                       | 16                                |
| <b>SAC</b>            | Buffer de Replay                               | 500,000 transições                |
|                       | Coeficiente de Entropia ( $\alpha$ )           | 0.1                               |
|                       | Taxa de Aprendizagem ( $lr$ )                  | $10^{-4}$                         |
|                       | Arquitetura da Rede                            | MLP [256, 256]                    |
| <b>Evolutivo (AE)</b> | Tamanho da População ( $K$ )                   | 30 genomas                        |
|                       | Taxa de Mutação ( $p_{mut}$ )                  | 0.1                               |
|                       | Desvio Padrão Inicial ( $\sigma$ )             | 0.1 (Decaimento: $\times 0.995$ ) |
|                       | Episódios de Avaliação por Genoma ( $E$ )      | 4                                 |
|                       | Dimensão Oculta da Rede ( $h$ )                | 32 ( $\approx$ 8k pesos)          |

#### 4.2.2. Espaço de Observação

As observações de cada agente são estritamente locais, parciais e expressas num referencial **egocêntrico** (relativo à orientação atual do próprio robô). O vetor de observação tem dimensão  $d_{obs} = 16 + (N - 1) \times 5 = 111$  (para  $N = 20$  agentes) e está estruturado em quatro componentes:

$$\mathbf{o}_i = [\mathbf{v}_{nest,i}, d_{nest,i}, \mathbf{l}_i, \mathbf{v}_{porta,i}, d_{porta,i}, \mathbf{n}_{1,i}, \dots, \mathbf{n}_{19,i}] \in \mathbb{R}^{111} \quad (4.1)$$

- (1) **Direção e Distância ao Ninho (4 valores):** Projeção do vetor direcional unitário do ninho nos eixos egocêntricos Frontal ( $F$ ), Direito ( $R$ ) e Superior ( $U$ ) do robô, mais a distância normalizada ( $d/2r_{arena}$ ). Esta informação de **homing** está

sempre disponível, mesmo quando existem muros entre o agente e o ninho. Este pressuposto é padrão em robótica de enxame — assume-se um sentido global de orientação para a base (análogo à bússola magnética dos pombos, à orientação solar das formigas ou ao GPS de drones), mas **nunca** um mapa prévio dos obstáculos. A parcialidade da observação é assim mantida ao nível dos obstáculos (detetáveis apenas localmente pelo LiDAR), e não ao nível do objetivo. Esta opção corrige uma limitação de uma versão anterior, em que a direção era anulada na ausência de linha de visão, o que cegava os agentes em todos os cenários com paredes e os impedia de aprender a contornar.

- (2) **LiDAR Horizontal (8 valores, vetor  $\mathbf{l}_i$ )**: Oito raios medidos em plano horizontal, uniformemente distribuídos a  $360^\circ$ , com alcance máximo de 5 m. O valor devolvido é  $1 - (d_{hit}/d_{max})$ , onde 1.0 significa impacto iminente.
- (3) **Direção e Distância à Porta/Entrada (4 valores,  $\mathbf{v}_{porta,i}$  e  $d_{porta,i}$ )**: Direção egocêntrica e distância normalizada à porta-objetivo do cenário, preenchidas **apenas** nos cenários com uma porta física definida (Porta Cooperativa); nos restantes são nulas. A porta é um sub-objetivo explícito do cenário — tal como o ninho —, pelo que a indicar não constitui revelar o trajeto. Distingue-se assim de fornecer *waypoints* das passagens dos labirintos, o que equivaleria a um planeador externo e quebraria a observabilidade parcial.
- (4) **Vetor de Vizinhos ( $19 \times 5 = 95$  valores, vetores  $\mathbf{n}_{j,i}$ )**: Para os restantes 19 agentes, envia-se o vetor de direção local (3 valores), distância normalizada (1 valor) e um sinal binário de comunicação (1 valor). Este último é ativado (1.0) quando o vizinho atinge o ninho e se encontra a repousar, servindo como mecanismo de recrutamento estigmérgico.

#### 4.2.3. Espaço de Ação e Dinâmica do Passo de Simulação

Cada agente emite, a cada passo, uma ação contínua  $\mathbf{a}_i = (a_F, a_R, a_U) \in [-1, 1]^3$  interpretada como um **deslocamento no referencial egocêntrico** do robô. O vetor é convertido para o referencial global através da base ortonormal  $\{F, R, U\}$  (Frontal, Direito, Superior) e escalado por um passo máximo  $\delta_{max} = 0.2$  m:

$$\Delta \mathbf{p}_i = 0.2 \cdot (a_F \mathbf{F}_i + a_R \mathbf{R}_i + a_U \mathbf{U}_i) \quad (4.2)$$

A orientação  $\mathbf{F}_i$  é atualizada para a direção do movimento sempre que  $\|\Delta \mathbf{p}_i\| > 0.02$ , garantindo coerência entre a pose visual e a trajetória.

O passo de simulação (**step**) processa o enxame por uma ordem determinística que assegura a consistência física:

- (1) **Perturbações dinâmicas**: movimento dos obstáculos e do alvo móvel; injeção de falhas de agentes (Secção 6.11) quando aplicável.
- (2) **Aplicação da ação com *wall-sliding***: cada deslocamento é projetado ortogonalmente à normal de qualquer muro intersetado, permitindo deslizamento em vez de paragem abrupta.

- (3) **Resolução de colisões:** reposicionamento por penetração contra obstáculos (distância euclidiana), muros (AABB) e fronteira circular da arena.
- (4) **Separação física e dispersão:** repulsão geométrica de pares sobrepostos e penalização *anti-clustering*, isenta nas zonas de cooperação.
- (5) **Lógica de tarefa:** verificação de ocupação do ninho ( $k_{min} \geq 3$ ), cerco do alvo móvel ( $360^\circ$ ) ou abertura da porta cooperativa.
- (6) **Atribuição de recompensa e terminação:** cálculo dos seis termos de  $\mathcal{R}$  e verificação do horizonte temporal ( $t \geq t_{max}$ ).

#### 4.2.4. Cenários de Teste

Para garantir uma avaliação exaustiva e progressiva das capacidades de controlo descentralizado, foram desenvolvidos seis cenários de dificuldade crescente (Figura 4.2):



FIGURA 4.2. Renderizações 3D dos seis cenários de teste, geradas a partir da geometria real do simulador (`scripts/render_maps.py`). Em cima, da esquerda para a direita: Sandbox, Beco Sem Saída (Muro em U) e Gargalo; em baixo: Quatro Salas, Porta Cooperativa (com a porta destacada a amarelo) e Percepção Cooperativa. A esfera verde assinala o ninho (ou o alvo móvel, no último cenário).

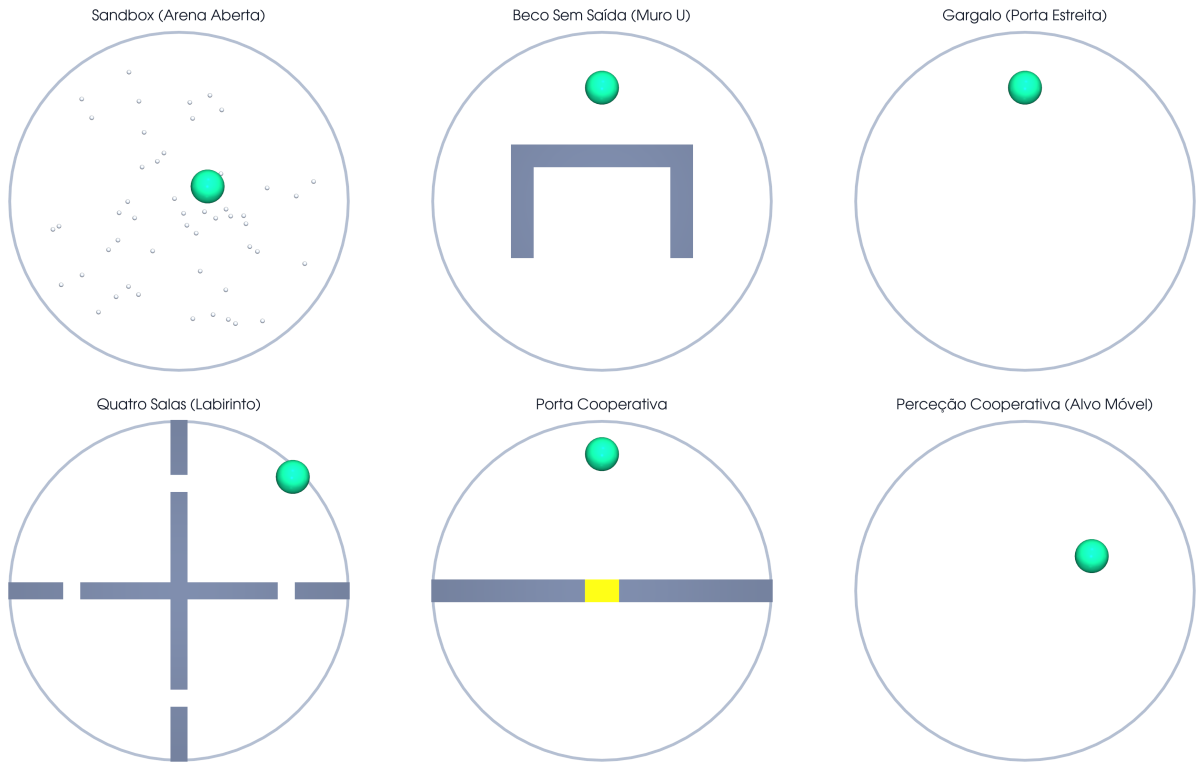


FIGURA 4.3. Vista de topo (planta) dos seis cenários, evidenciando a geometria das paredes e a largura das passagens. Mesma ordem da Figura 4.2: em cima, Sandbox, Muro em U e Gargalo; em baixo, Quatro Salas, Porta Cooperativa e Percepção Cooperativa.

- (1) **Foraging Aberto (Sandbox):** Cenário base sem barreiras fixas. Contém os agentes, obstáculos dinâmicos e um ninho móvel. Serve para aferir a capacidade pura de cooperação e gestão de energia sem entropia espacial acrescida.
- (2) **Beco Sem Saída (Muro em U):** Um grande obstáculo em forma de "U" bloqueia a linha direta ao ninho. A parede superior tem 14 m e as pernas laterais têm 10 m. Os agentes começam do lado de fora da "taça" e têm de explorar lateralmente as aberturas de 7 m. Avalia a exploração prolongada em detrimento da navegação gananciosa.
- (3) **Gargalo (Porta Estreita):** A arena é cortada a meio por uma parede contínua, deixando apenas uma passagem de 1.5 m de largura. Obriga a um alinhamento e gestão do fluxo do enxame, punindo gravemente o congestionamento desenfreado.
- (4) **Quatro Salas (Labirinto):** Duas grandes paredes cruzam a arena vertical e horizontalmente, criando quatro salas isoladas ligadas apenas por corredores laterais de 1.5 m de largura. Os agentes começam no extremo oposto ao ninho, exigindo memória temporal e navegação estruturada de longo termo.
- (5) **Porta Cooperativa:** Similar ao Gargalo, mas a passagem de 3 m de largura encontra-se bloqueada por um portão físico. A porta só abre (deslizando para fora do caminho, com realce visual no *dashboard*) quando no mínimo 3 agentes ocupam a zona de pressão em simultâneo. Por exigir uma sincronização mais demorada,

este cenário dispõe de um horizonte temporal alargado de 800 passos (face aos 500 dos restantes), dando margem para que o enxame coordene a abertura e ainda alcance o ninho. Foca-se em comportamentos de sincronização sem comunicação explícita.

- (6) **Perceção Cooperativa (Alvo Móvel):** A noção de "ninho físico" desaparece. O alvo de interesse viaja livremente pelo mapa ao dobro da velocidade normal (0.03 m/s). Só é considerado capturado quando 3 agentes se aproximam a menos de 4 m de distância e distribuem-se num raio de 360° em redor do mesmo, testando manobras de cerco e perseguição.

### 4.3. Design da Função de Recompensa

Para guiar a emergência de comportamentos coordenados e seguros, a função de recompensa global  $\mathcal{R}$  partilhada entre os agentes foi formulada para garantir sinais de treino densos (mitigando problemas de exploração [27]) e a rigorosa penalização de infrações físicas:

$$\mathcal{R}(s, \mathbf{a}) = R_{\text{tarefa}} + R_{\text{progresso}} + R_{\text{exploração}} - R_{\text{dispersão}} - R_{\text{colisão}} - R_{\text{controlo}} \quad (4.3)$$

Os valores exatos utilizados na simulação estão calibrados da seguinte forma:

- **Tarefa ( $R_{\text{tarefa}}$ ):** Recompensa majorada atribuída *apenas* quando a colaboração é bem sucedida. No *foraging* normal, +100 quando  $k_{\min} \geq 3$  repousam no ninho em simultâneo. No modo de Perceção Cooperativa, o valor aumenta para +300 após um cerco de sucesso de 360°.
- **Progresso ( $R_{\text{progresso}}$ ):** Baseado no método *Potential-Based Reward Shaping* (PBRs) [33], atribui  $10.0 \cdot (\Phi_{t-1} - \Phi_t)$ , onde  $\Phi$  é o **potencial de navegação** até ao ninho. Nos cenários com paredes (Muro U, Gargalo, Quatro Salas e Porta Cooperativa) este potencial é a **distância geodésica** — o comprimento do caminho mais curto que contorna os obstáculos, calculado por uma pesquisa de custo uniforme (Dijkstra, 8-conexa) sobre uma grelha de discretização (0.4 m, com as paredes dilatadas pelo raio do robô) — e não a distância euclidiana. Esta escolha elimina na raiz um mínimo local clássico: com a distância euclidiana, contornar uma parede *afasta* o agente do ninho e é penalizado, prendendo o enxame contra os obstáculos; com a distância geodésica, qualquer passo ao longo do caminho viável é recompensado. Por se tratar de PBRs, não altera a política ótima do cenário [33]. Decisivamente, o campo geodésico é um **senal de treino** (a geometria é conhecida pelo simulador apenas durante o treino) e **não** uma observação: em execução o agente dispõe somente da bússola de *homíng* euclidiana e do LiDAR local (Secção 4.2.2), preservando a observabilidade parcial. Distingue-se assim, claramente, de fornecer ao agente um mapa dos obstáculos ou *waypoints* de um planeador A\*.

- **Exploração** ( $R_{\text{exploração}}$ ): Bónus de exploração *count-based* que atribui +2.0 na primeira vez que um agente visita cada célula de uma grelha de discretização espacial ( $2 \times 2$  m). Complementa o termo de progresso geodésico, premiando a cobertura de zonas novas em regiões onde o gradiente do potencial é pouco informativo (por exemplo, grandes áreas abertas ou antes de o agente detetar localmente um desvio), em linha com a necessidade de mecanismos densos de exploração identificada por Xiao et al. (2026) [27].
- **Dispersão** ( $R_{\text{dispersão}}$ ): Penalização *anti-clustering* de até  $-1.5$  por cada par de agentes que se aproxime a menos de 1.0 m, proporcional à sobreposição. Desincentiva o colapso do enxame numa única “bola” compacta (comportamento degenerado observado em políticas com parâmetros partilhados, em que observações semelhantes geram ações idênticas). A penalização é **isenta nas zonas de cooperação** (a menos de 4 m do ninho, do alvo móvel ou da zona de pressão da porta), onde a tarefa exige precisamente a agregação de  $k_{\min} \geq 3$  agentes.
- **Colisão e Fome** ( $R_{\text{colisão}}$ ): A colisão com **obstáculos** (parede, obstáculo móvel ou delimitação exterior da arena) é penalizada com  $-2.0$  por passo e resolvida por reposicionamento (*push-out* pela profundidade de penetração; AABB nos muros, distância euclidiana nos obstáculos esféricos). A colisão física **entre agentes** (sobreposição a menos de  $2r_{\text{robot}}$ ) é simultaneamente **resolvida** por separação física (repulsão geométrica que impede interpenetrações) e **penalizada** de forma proporcional à penetração ( $\sim -1.0$ ), ensinando os agentes a não chocar. Esta penalização distingue-se da *dispersão* (acima): a dispersão pune a mera *proximidade* (0.3–1.0 m) e apenas na navegação, ao passo que a penalização de colisão pune a *sobreposição* ( $< 0.3$  m) e aplica-se **sempre**, mesmo nas zonas de cooperação (chocar é indesejável em qualquer contexto). Adicionalmente, caso o temporizador de *hunger* ultrapasse 250 passos, o agente “morre”, sofre um revés crítico de  $-50.0$  e a sua posição é restabelecida. Esta reposição está **ativa apenas no forrageamento contínuo** (cenário Sandbox); nos cenários de objetivo fixo foi desativada, pois a combinação do teletransporte com a re-sincronização do potencial tornava o termo de progresso *farmável* (aproximar-se, ser repostado longe sem cobrança do recuo, repetir), induzindo políticas degeneradas que acumulavam recompensa de *shaping* sem nunca cumprir a tarefa.
- **Controlo Energético** ( $R_{\text{controlo}}$ ): Subtração de  $-0.05$  a cada passo de simulação por agente não inativo. Pressiona temporalmente o enxame para finalizar o forrageamento o mais célere possível.

#### 4.4. Desafios de Engenharia e Otimização Computacional

A construção de um simulador de raiz capaz de suportar o treino de algoritmos de aprendizagem por reforço profundo exige um rigoroso esforço de engenharia de software. O ambiente desenvolvido (`swarm_env_3d.py` e o respetivo visualizador `launcher_dashboard.py`)

foi desenhado para mitigar estrangulamentos computacionais comuns em arquiteturas *Multi-Agent*, focando-se na otimização matemática e abstração arquitetural.

#### 4.4.1. Motor de Visualização Interativa (Dashboard)

Para permitir a inspeção do comportamento emergente do enxame sem comprometer o desempenho do treino, o sistema adota um padrão de arquitetura que desacopla rigorosamente a lógica de simulação física do motor de renderização gráfica. A inspeção do comportamento é feita por um visualizador 3D implementado com o motor **Ursina** (assente em Panda3D), que projeta a telemetria do motor contínuo interno num espaço tridimensional navegável, com representação do ninho, obstáculos móveis, muros e indicadores de proximidade. O controlo experimental — lançamento de treinos, seleção de cenários e geração de gráficos — é centralizado num *dashboard* interativo construído em **CustomTkinter**. Esta separação permite que o simulador opere num modo *headless* (sem sobrecarga gráfica) durante sessões de treino intensivo — processando milhares de transições por segundo —, enquanto oferece um modo interativo para demonstração e análise qualitativa das políticas aprendidas.

#### 4.4.2. Otimização Matemática da Física e Detecção de Colisões

A viabilidade do RL depende de ciclos de simulação contínuos em ordem de milissegundos. Simular colisões dinâmicas para  $N = 20$  agentes e uma centena de obstáculos não-estáticos de forma ineficiente levaria a um aumento quadrático no custo computacional.

- **Detecção Geométrica de Colisões:** O motor resolve as colisões por testes geométricos diretos — distância Euclidiana para os obstáculos esféricos e caixas envolventes alinhadas aos eixos (*Axis-Aligned Bounding Boxes*, AABB) para os muros retangulares —, recorrendo a operações vetoriais **NumPy** no cálculo de distâncias e profundidades de penetração. Um passo dedicado de **separação física** repele geometricamente qualquer par de agentes sobrepostos, assegurando a ausência de interpenetrações não-físicas no enxame.
- **Wall-Sliding Cinemático:** Quando um robô entra em contacto com uma parede retangular, o motor não o imobiliza mecanicamente (o que penalizaria irreversivelmente a exploração do algoritmo MARL). Em vez disso, recorre à projeção ortogonal do vetor de velocidade ao longo da direção normal da superfície de colisão, reproduzindo matematicamente de forma realista o deslizamento (*wall-sliding*) contínuo de um chassi móvel *differential-drive*.

#### 4.4.3. Mecanismo de Atenção sobre Vizinhos (GNN)

O extrator de relações entre robôs foi implementado de raiz em **PyTorch puro** (sem recurso a bibliotecas de grafos como o PyTorch Geometric), através de um mecanismo de **atenção de produto interno escalado** sobre o conjunto de vizinhos, conforme formalizado nas Equações 2.18–2.20.

- **Representação Densa do Vizinhança:** A cada passo, cada agente  $i$  observa os restantes  $N - 1$  agentes através de um vetor de *features* de dimensão fixa (direção egocêntrica, distância normalizada e sinal de comunicação por vizinho). Diferentemente de uma abordagem baseada em arestas esparsas com raio de comunicação fixo, todos os vizinhos são apresentados ao módulo de atenção, sendo a **relevância de cada um aprendida** pelos coeficientes  $\alpha_{ij}$  — a rede atribui dinamicamente pesos próximos de zero aos vizinhos irrelevantes, dispensando um corte geométrico explícito.
- **Dimensão Fixa e *Parameter Sharing*:** Como a representação tem dimensão fixa ( $\mathbb{R}^{111}$ ), a integração em *batches* de treino é direta, sem necessidade de *padding* dinâmico ou conversão para listas de índices. As projeções *Query-Key-Value* e o *softmax* operam sobre o número de vizinhos presentes, pelo que a mesma rede processa enxames de qualquer dimensão — viabilizando o *Zero-Shot Transfer* para  $N \in \{10, 50, 100\}$  sem retreino.

#### 4.4.4. Ambientes Vetorizados e Paralelização Multiprocesso

Por forma a escalar exponencialmente a recolha de experiência (nas instâncias SAC/PPO) e na avaliação paralela da neuroevolução:

- **Vectorized Environments:** O motor de ambiente Gym original foi encapsulado através dos *wrappers* `SubprocVecEnv` (fornecidos pela \*Stable-Baselines3\*), forçando a inicialização de dezenas de cópias do simulador correndo de forma isolada e simultânea através de múltiplos processos de CPU paralelos.
- **Mecanismo de Guilhotina (Early Stopping):** Este mecanismo heurístico otimizou criticamente o tempo de otimização estocástica evolutiva. Se as recompensas operacionais de um indivíduo descenderem subitamente até limiares proibitivos nos passos primários da simulação — fruto de uma política disfuncional gerada por mutação —, a execução contínua é descartada, transitando de imediato os recursos de ciclo de CPU do sistema para o processamento de novos indivíduos concorrentes na mesma geração.

## Arquitetura Algorítmica e Controladores

### 5.1. Algoritmo 1: Multi-Agent Reinforcement Learning (RS2C)

A primeira abordagem investigada fundamenta-se no *framework* **Robust and Scalable Swarm Control (RS2C)**. Este adota um esquema de **execução descentralizada com partilha de parâmetros** (*parameter sharing*) — em que todos os agentes partilham uma única política que atua sobre observações estritamente locais —, formulando a coordenação do enxame como um Processo de Decisão de Markov Parcialmente Observável Cooperativo (Dec-POMDP):

$$\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i \in \mathcal{N}}, \mathcal{P}, \mathcal{R}, \Omega, \{\mathcal{O}_i\}_{i \in \mathcal{N}}, \gamma \rangle \quad (5.1)$$

Sob o RS2C investigam-se dois algoritmos de aprendizagem por reforço profundo padrão: **PPO** (*on-policy*) e **SAC** (*off-policy*). Em ambos, os  $N$  agentes partilham uma única política (*parameter sharing*) que recebe a observação local de dimensão fixa  $\mathbf{o}_i \in \mathbb{R}^{111}$  — a qual já codifica, em formato *dense*, as *features* de toda a vizinhança (Secção 4.2.2). Na implementação atual, esta política partilhada é uma rede totalmente ligada (MLP [256, 256], da *Stable-Baselines3*); o **mecanismo de atenção sobre grafo** formalizado no Capítulo 2 é instanciado no controlador neuroevolutivo (Secção 5.2), sendo a base da invariância à permutação e da capacidade de *Zero-Shot Transfer* aí avaliadas. A integração direta do módulo de atenção numa política treinável por gradiente constitui uma extensão natural deixada para trabalho futuro (Secção 7.2).

#### 5.1.1. Configuração PPO (*On-Policy*)

No modo PPO, os pesos da política MLP partilhada são otimizados através de gradientes calculados sobre *rollouts* de experiência recolhidos com a política atual. O *pipeline* de treino opera da seguinte forma:

- (1) **Recolha de Experiência:** Em cada iteração,  $N_{arenas} = 16$  cópias paralelas do simulador (`SubprocVecEnv`) geram transições durante  $n_{steps} = 64$  passos cada. Como os  $N = 20$  agentes de cada arena são tratados como ambientes independentes (*parameter sharing*), o *rollout buffer* acumula  $16 \times 20 \times 64 = 20\,480$  transições por iteração.
- (2) **Cálculo da Vantagem:** As vantagens  $\hat{A}_t$  são estimadas via GAE (Equação 2.11) com  $\gamma = 0.99$  e  $\lambda = 0.95$  sobre o *rollout* completo.
- (3) **Atualização:** O buffer é amostrado em *mini-batches* de 512 transições e o gradiente de  $L^{\text{PPO}}$  é calculado durante  $n_{epochs} = 4$  passagens completas.

- (4) **Parameter Sharing:** Os  $N = 20$  robôs partilham o mesmo conjunto de parâmetros  $\theta$ . Cada transição  $(o_i^t, a_i^t, r_t)$  de qualquer agente contribui para o gradiente, aumentando efetivamente o tamanho do *batch* por fator  $N$ .

### 5.1.2. Configuração SAC (*Off-Policy*)

O SAC opera em modo *off-policy*: as experiências acumulam-se assincronamente num **Replay Buffer** de capacidade  $|\mathcal{D}| = 500\,000$  transições. Para  $N = 20$  agentes, cada passo de simulação contribui 20 transições, preenchendo o buffer rapidamente. O SAC mantém dois críticos independentes  $Q_{\psi_1}$  e  $Q_{\psi_2}$  (*double Q-trick*) e a cada passo do ambiente uma mini-batch de 256 transições é amostrada aleatoriamente para atualizar os parâmetros. O coeficiente de temperatura  $\alpha = 0.1$  é mantido constante, sem adaptação automática.

### 5.1.3. Representação da Vizinhança e *Parameter Sharing*

A cada passo de simulação, o vetor de observação de cada agente é construído em `swarm_env_3d.py`:

- (1) **Vizinhança Completa:** As *features* de todos os restantes  $N - 1$  agentes (direção egocêntrica, distância normalizada e sinal de comunicação) são incluídas no vetor de observação  $\mathbf{o}_i$ . Não é aplicado um corte por raio de comunicação fixo: a seleção dos vizinhos relevantes é delegada no mecanismo de atenção, que aprende os pesos  $\alpha_{ij}$ .
- (2) **Representação *Dense* de Dimensão Fixa:** O vetor  $\mathbf{o}_i \in \mathbb{R}^{111}$  incorpora os  $N - 1$  vizinhos em formato *dense*, preservando uma dimensão de entrada constante e dispensando estruturas esparsas (*edge-index*) ou *padding* dinâmico.
- (3) **Dimensão de Entrada Fixa (PPO/SAC):** A política MLP partilhada recebe um vetor de dimensão **fixa**  $\mathbb{R}^{111}$ , definida para  $N = 20$  agentes. Por construção, esta abordagem **não é invariante** ao tamanho do exame: alterar  $N$  modifica a dimensão da observação  $(16 + (N - 1) \times 5)$  e inviabiliza a aplicação direta da rede. A invariância à permutação e o *Zero-Shot Transfer* para  $N \in \{10, 50, 100\}$  são propriedades do mecanismo de atenção (Equações 2.18 e 2.19), cujas operações QKV e *softmax* se definem sobre o número de vizinhos presentes; são, por isso, avaliados no controlador neuroevolutivo (Secção 5.2).

## 5.2. Algoritmo 2: Otimização Bio-inspirada (Benchmark)

A *baseline* adotada assenta num processo de **Neuroevolução** [9], utilizando um Algoritmo Evolutivo clássico com elitismo (preservação dos 20% melhores) e mutações Gaussianas independentes nos pesos ( $p_{mut} = 0.1$ , com atenuação *sigma decay*). É neste controlador que se instancia a **rede de grafos com atenção** (GNNAgent3D) formalizada no Capítulo 2: é precisamente esta arquitetura, invariante ao número de agentes, que habilita o *Zero-Shot Transfer*. A otimização evolutiva *offline* ajusta os seus pesos sobre 30 genomas candidatos, sem recurso a gradientes nem a um sinal de recompensa denso por passo.

### 5.2.1. Ciclo de Treino Evolutivo

O treino evolutivo segue o ciclo *avaliação*  $\rightarrow$  *seleção*  $\rightarrow$  *mutação*, implementado em `evo_trainer_3d.py`:

- (1) **Avaliação Paralela:** Os  $K = 30$  genomas são avaliados em paralelo via `multiprocessing.Pool` com até 16 processos (limitado ao mínimo entre o tamanho da população e o número de núcleos disponíveis). Cada processo executa  $E = 4$  episódios independentes (com *seeds* fixas ao longo da evolução) e devolve  $J(\theta_k)$  (Equação 2.1).
- (2) **Mecanismo de Guilhotina:** Se a recompensa acumulada após 150 passos for inferior ao limiar  $J < -200$ , o episódio é terminado prematuramente e o genoma recebe uma penalidade adicional de  $-1000$ . Este mecanismo evita desperdiçar ciclos de CPU em genomas claramente disfuncionais.
- (3) **SeleçãoELITista:** Os  $e = 6$  melhores genomas são copiados intactos. Os restantes 24 são gerados por mutação element-wise (Equação 2.2) de cópias aleatórias dos elites.
- (4) **Anilamento de  $\sigma$ :** A cada geração,  $\sigma$  é atualizado pela Equação 2.3, com decaimento de 0.5% por geração.
- (5) **Critério de Paragem:** O treino prossegue até esgotar o orçamento temporal configurado (e.g., 120 minutos), garantindo duração determinística e comparável com os algoritmos MARL.

### 5.3. Plano de Experiências e Tratamento Estatístico

O *pipeline* experimental é segmentado em cinco avaliações prioritárias (Tabela 5.1) por forma a assegurar validação e significância dos resultados reportados na tese.

TABELA 5.1. Plano de testes experimentais e métricas associadas.

| Prior. | Experiência                                           | Duração     | Objetivo Científico                                           | Métrica             |
|--------|-------------------------------------------------------|-------------|---------------------------------------------------------------|---------------------|
| 1      | Treino longo (Sandbox)                                | 24h         | Convergência máxima dos 3 modelos no cenário base.            | $P_{task}$ (Treino) |
| 2      | Avaliação Determinística ( <code>eval_all.py</code> ) | 5 min       | Desempenho sem exploração (20 episódios).                     | $P_{task}$ (Eval)   |
| 3      | Benchmark Estatístico (30 runs)                       | 3-4 dias    | Validação de hipóteses estatísticas ( $p < 0.05$ ).           | Teste $t$ -Student  |
| 4      | Robustez a Falhas (Remover 10% agentes)               | 1h + Treino | Medir resiliência à perda súbita de robôs a meio do episódio. | $R_{robust}$        |
| 5      | Escalabilidade Zero-Shot ( $N \in \{10, 50, 100\}$ )  | 3h          | Testar generalização do grafo dinâmico sem re treino.         | $S_{scale}$         |

A integridade empírica apoia-se na execução paralela de simulações heterogêneas. Quaisquer evidências de supremacia algorítmica serão escrutinadas estatisticamente recorrendo ao teste *t*-Student, assumindo o valor convencional de significância estatística em inteligência artificial ( $p < 0.05$ ).

## Resultados Experimentais e Discussão

Este capítulo é dedicado à apresentação e análise rigorosa de todos os dados recolhidos durante as baterias de simulação. Aqui, confrontamos o paradigma de *Multi-Agent Reinforcement Learning* (PPO e SAC) com o paradigma de Otimização Bio-inspirada (Algoritmos Evolutivos), utilizando tratamento estatístico para validar as nossas hipóteses de investigação.

### 6.1. Metodologia de Avaliação e Notas de Leitura

Antes da análise, importa fixar as convenções de leitura dos resultados:

- **Runs independentes:** Cada combinação (algoritmo, cenário) foi treinada em  $R$  *runs* independentes com sementes distintas. As curvas reportam a média e a banda sombreada corresponde ao desvio padrão entre *runs*.
- **Eixo temporal normalizado:** Como o GNN (evolutivo) e os métodos MARL (PPO/SAC) operam em unidades de progresso distintas (gerações  $\times$  passos vs. *timesteps* de política), o eixo das abcissas das curvas comparativas é normalizado para o intervalo  $[0\%, 100\%]$  do orçamento de treino de cada algoritmo, permitindo uma sobreposição justa.
- **Métrica de tarefa pura ( $P_{task}$ ):** Para além da recompensa total (que inclui *shaping*), reporta-se a recompensa de tarefa pura (recolhas  $\times 100$ ), obtida em avaliação determinística, por ser a medida não-enviesada do sucesso na missão.
- **Escalas não-comparáveis entre paradigmas:** O GNN usa *fitness* evolutiva e o PPO/SAC recompensa episódica; comparações de valor absoluto entre paradigmas devem ser feitas com cautela, privilegiando a métrica  $P_{task}$  da avaliação determinística.

### 6.2. Desempenho de Tarefa por Cenário ( $P_{task}$ )

A métrica central de comparação é o desempenho de **tarefa** em avaliação determinística: a taxa de sucesso ( $P_{task}$ , fração de episódios com pelo menos uma recolha) e o número de recolhas por episódio. Ao contrário da recompensa de treino — que inclui *shaping* e mistura escalas entre paradigmas (*fitness* evolutiva vs. recompensa episódica) —, estas métricas são diretamente comparáveis entre os três algoritmos e medem o cumprimento efetivo da missão. Os valores reportados resultam de **30 episódios determinísticos emparelhados** (*seeds* comuns aos três algoritmos) sobre os modelos finais.

A Tabela 6.2 e as Figuras 6.1–6.2 revelam três perfis de desempenho nitidamente distintos. O **SAC** é o único algoritmo a resolver os seis cenários (100% de sucesso em todos,

incluindo o Muro U, onde nenhum outro paradigma navega de forma fiável), confirmando-se como o controlador mais robusto em navegação. O **PPO** iguala o SAC em cinco cenários — superando-o inclusive em magnitude de recolhas no Sandbox (73,3 vs. 25,2 recolhas/ep) e na Porta Cooperativa (67,7 vs. 60,5) —, mas **colapsa para 0% no Muro U**: a recompensa episódica registada nesse cenário ( $\approx 132$  mil, a mais elevada de toda a campanha, acompanhada de zero recolhas) denuncia um caso claro de *reward hacking*, em que a política maximiza o bônus de exploração percorrendo indefinidamente o beco sem jamais recolher comida (Secção 6.13).

O **GNN evolutivo** fica muito aquém na tarefa: só atinge sucesso assinalável nos cenários sem paredes interiores (Sandbox 66,7%, Perceção Cooperativa 96,7%) e ainda assim com magnitude residual (0,77 e 2,63 recolhas/ep), falhando por completo (0%) os três cenários com gargalos estreitos — Gargalo, Quatro Salas e Porta Cooperativa. Esta dissociação entre *fitness* de treino elevada e recolhas quase nulas é a assinatura do fenómeno de *fitness exploitation* analisado adiante. Curiosamente, no Muro U o GNN (43,3%) chega a superar o PPO (0%), não por resolver o labirinto, mas por não sucumbir ao mesmo *reward hacking*. O padrão transversal é claro: a fronteira de dificuldade não separa cenários “cooperativos” de “simples”, mas antes os cenários de **navegação por gargalos** dos de espaço aberto — e é precisamente nessa classe que se observa a maior divergência entre paradigmas.

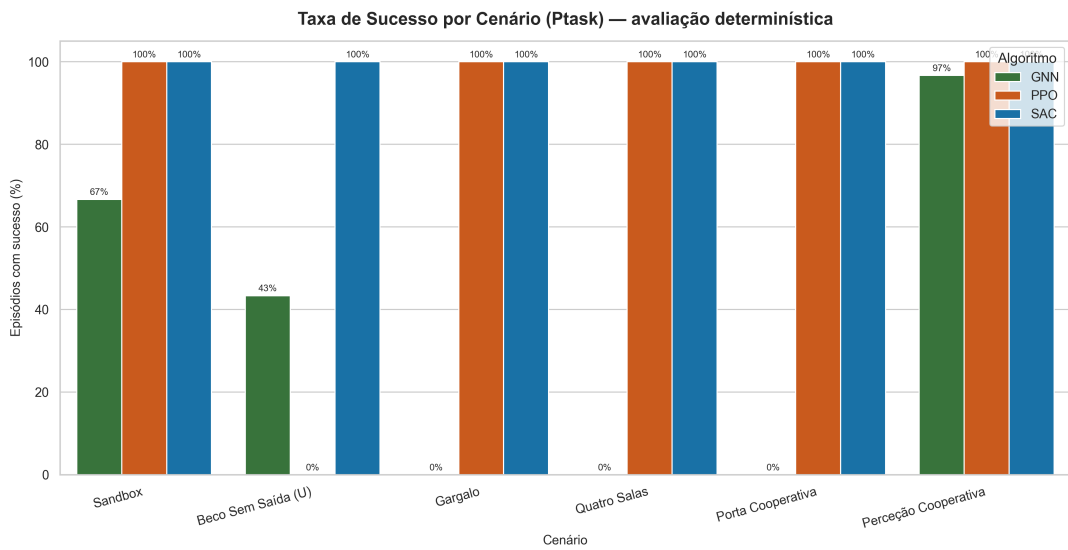


FIGURA 6.1. Taxa de sucesso ( $P_{task}$ ) por cenário, em avaliação determinística (30 episódios emparelhados por algoritmo).

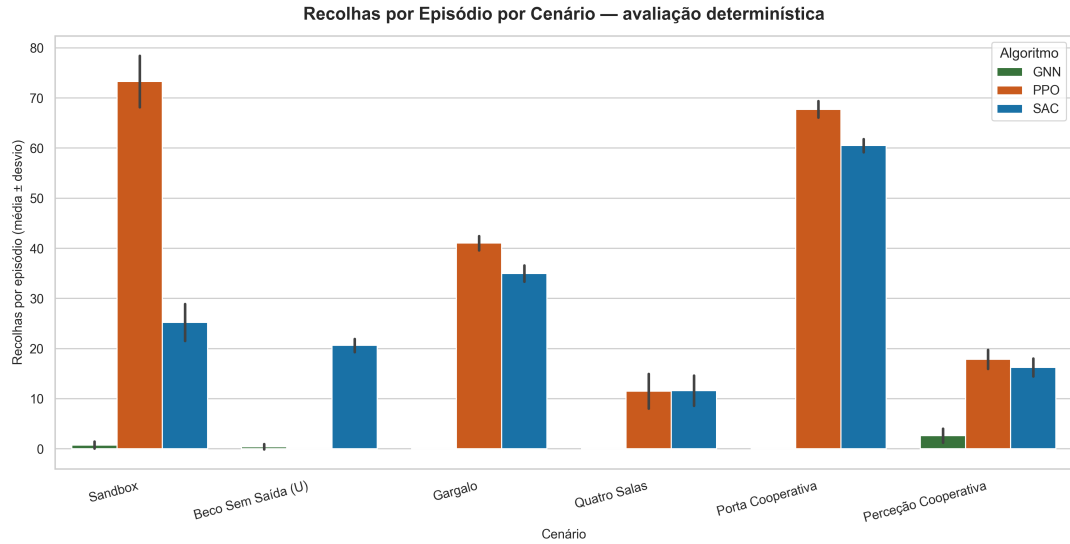


FIGURA 6.2. Recolhas por episódio (média  $\pm$  desvio padrão) por cenário. Mede a magnitude do desempenho, para além do sucesso binário.

### 6.3. Análise Qualitativa da Navegação: Mapas de Calor

Para fundamentar a opção pelo potencial geodésico no termo de progresso e diagnosticar o comportamento de navegação, recorre-se a dois tipos de mapa de calor. A Figura 6.3 contrasta o potencial euclidiano com o geodésico no Muro U: as curvas de nível euclidianas atravessam a parede (atraindo o agente para o interior do beco), enquanto as geodésicas a contornam, atribuindo gradiente positivo ao desvio correto. A Figura 6.4 mostra a densidade de ocupação dos robôs — zonas brilhantes coladas a uma parede revelam agentes presos, ao passo que um fluxo que contorna o obstáculo indica navegação resolvida.

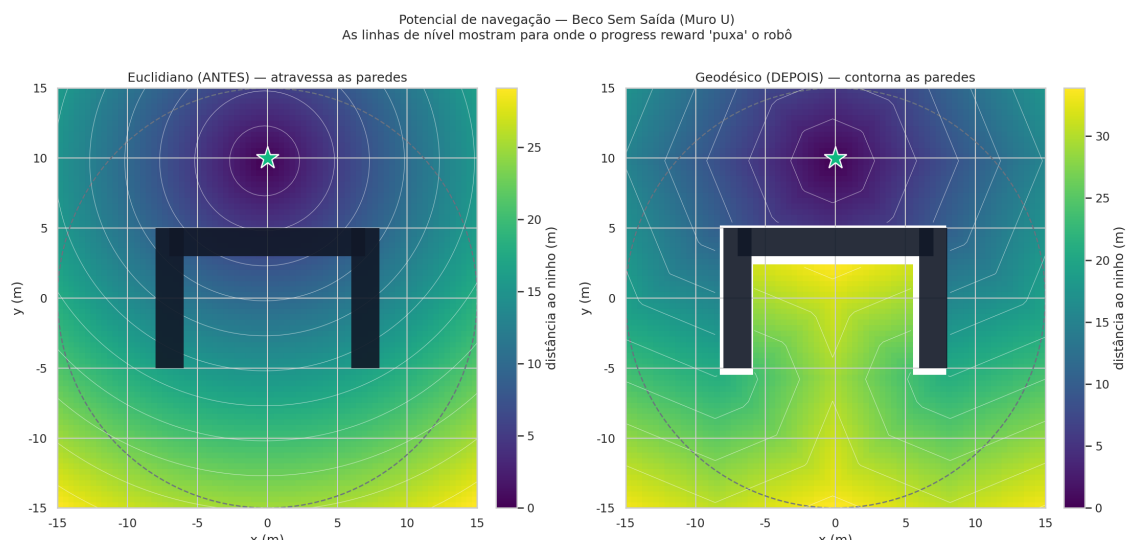


FIGURA 6.3. Potencial de navegação no Muro U: euclidiano (esquerda) vs. geodésico (direita). As curvas de nível indicam a direção para a qual o termo de progresso atrai o agente; só o geodésico contorna a barra do U.

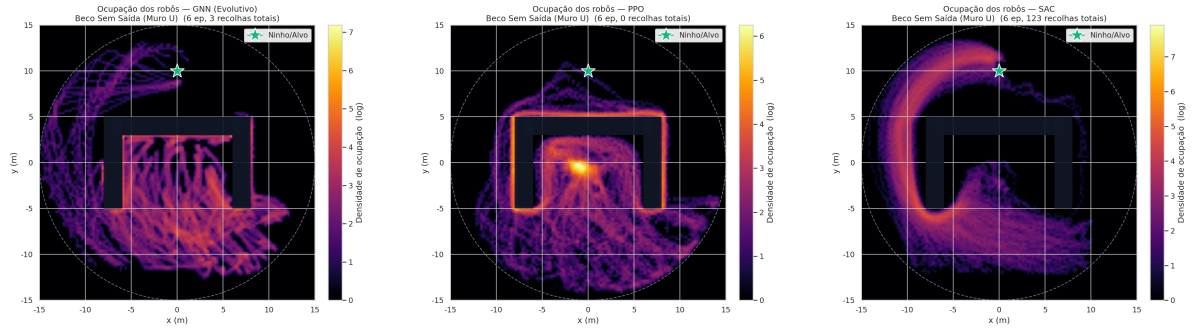
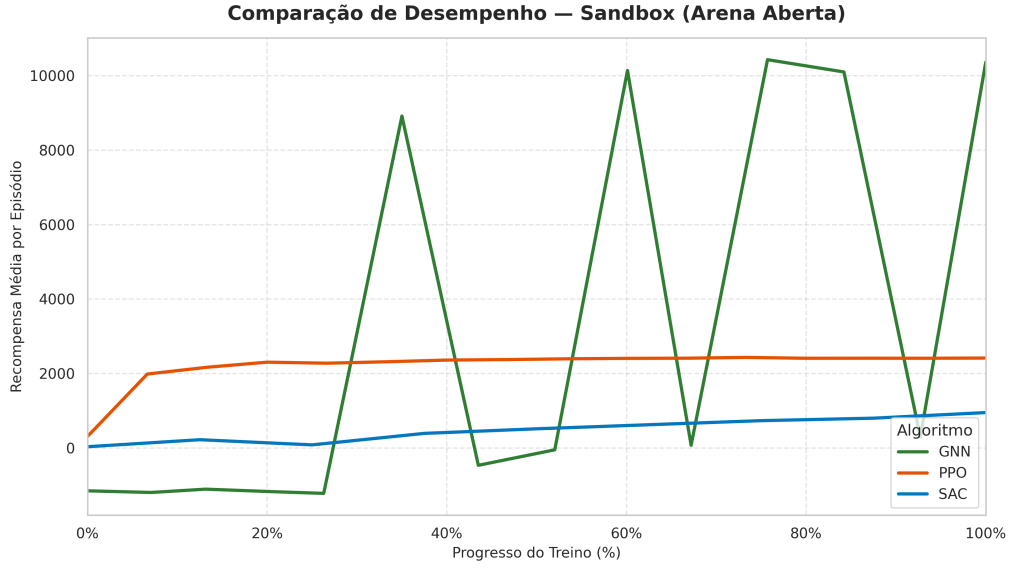


FIGURA 6.4. Densidade de ocupação dos robôs no Muro U (GNN, PPO e SAC, da esquerda para a direita). O fluxo que contorna a barra do U pelos lados evidencia a navegação resolvida pelo potencial geodésico.

#### 6.4. Convergência e Desempenho no Cenário Base (Sandbox)

O cenário Sandbox — arena aberta, sem paredes interiores — isola a capacidade de *foraging* da componente de navegação por obstáculos, servindo de referência de otimalidade. A Figura 6.5 mostra que os dois métodos de gradiente convergem para políticas funcionais, mas com dinâmicas distintas: o **SAC** (*off-policy*) sobe rapidamente nos primeiros passos, beneficiando da reutilização de experiência do *replay buffer*; o **PPO** (*on-policy*) progride mais lentamente mas atinge o desempenho de tarefa mais alto (73,3 recolhas/ep, contra 25,2 do SAC). O **GNN evolutivo** estabiliza num patamar muito inferior: apesar de alcançar sucesso em dois terços dos episódios, recolhe em média menos de uma unidade por episódio, sinal de uma política que aprende a aproximar-se da comida mas raramente completa o ciclo de recolha-e-entrega.



ível e há obstáculos dinâmicos. Avalia a capacidade base de forrageamento e orientação. | Eixo X normalizado (0%→100% do treino de cada algoritmo) para comparação justa. GNN: fit

FIGURA 6.5. Curvas de aprendizagem dos três algoritmos no cenário Sandbox. O eixo das abscissas está normalizado (0–100% do treino). Cada curva corresponde a uma única execução de treino (ver Secção 6.6).

A Tabela 6.1 sintetiza o desempenho de avaliação neste cenário.

TABELA 6.1. Desempenho no cenário Sandbox (avaliação determinística, 30 episódios; recolhas em média  $\pm$  desvio padrão entre episódios).

| Algoritmo       | Recolhas/ep      | $P_{task}$ | Recompensa (eval) |
|-----------------|------------------|------------|-------------------|
| GNN (Evolutivo) | $0,77 \pm 0,68$  | 66,7%      | 413               |
| PPO             | $73,30 \pm 5,14$ | 100%       | 49 183            |
| SAC             | $25,23 \pm 3,69$ | 100%       | 19 533            |

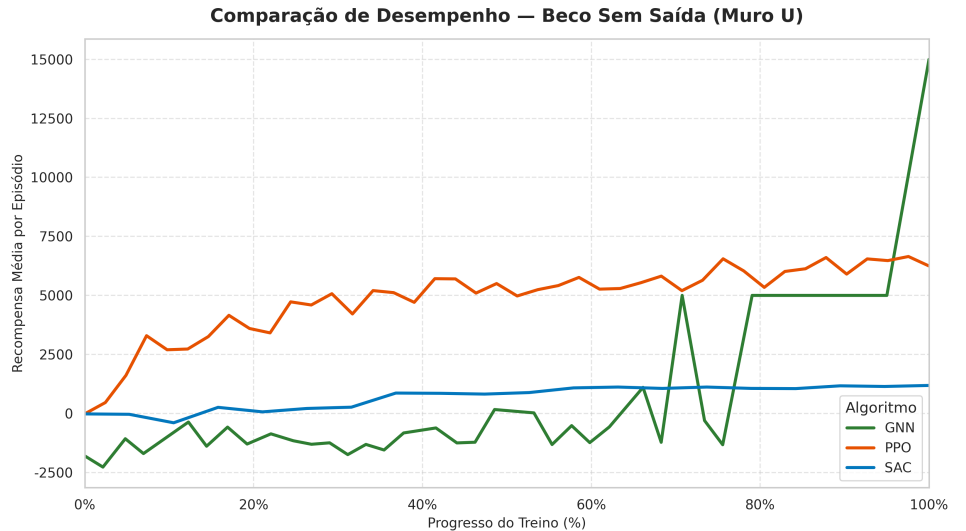
A coluna “Recompensa (eval)” refere-se à recompensa episódica média; não é comparável entre o GNN (cuja seleção usa *fitness*) e os métodos de gradiente, servindo apenas de leitura intra-algoritmo.

## 6.5. Desempenho Comparativo por Cenário (Curvas de Aprendizagem)

As curvas de aprendizagem por cenário (Figuras 6.6–6.10) detalham a dinâmica de treino subjacente aos resultados de tarefa. Importa uma ressalva de leitura: os eixos verticais não são comparáveis entre paradigmas — o GNN é selecionado por *fitness* (dominada pelo termo de comida  $\times 10^4$ ), ao passo que PPO e SAC reportam recompensa episódica com *shaping* —, pelo que cada subgráfico deve ser lido como a trajetória *interna* de cada algoritmo, e não como uma corrida no mesmo eixo.

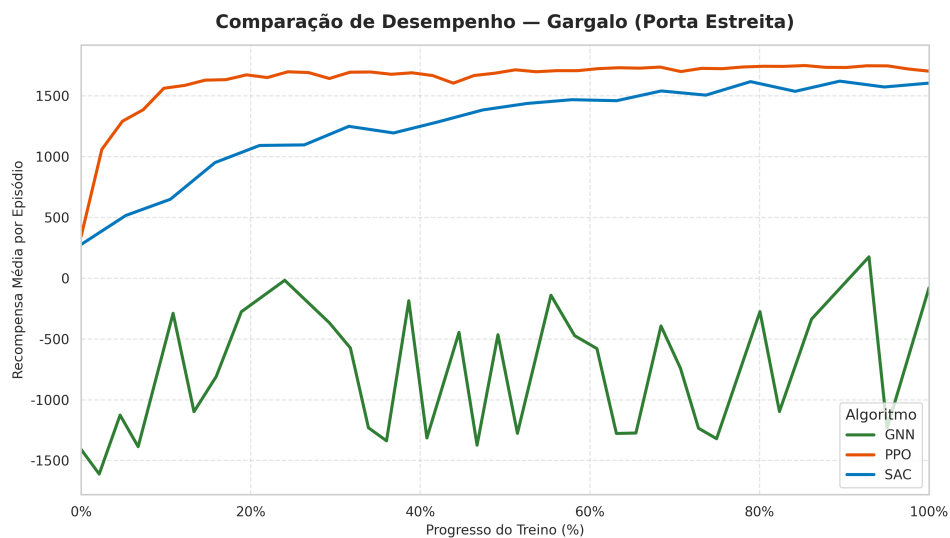
Nos cenários de gargalo (**Gargalo**, Figura 6.7; **Quatro Salas**, Figura 6.8; **Porta Cooperativa**, Figura 6.9) observa-se o padrão dominante do estudo: o SAC sobe cedo e estabiliza em valores elevados, o PPO converge mais tarde para desempenho equivalente, e o GNN estagna — a sua *fitness* sobe pela componente de *shaping* geodésico sem que

tal se traduza em recolhas, exatamente o sintoma de *fitness exploitation*. No **Muro U** (Figura 6.6), a curva do PPO é enganadora: a recompensa dispara para valores recorde enquanto a tarefa permanece a zero, materializando o *reward hacking* do bônus de exploração. Já na **Percepção Cooperativa** (Figura 6.10), cenário de alvo móvel sem paredes interiores, os três algoritmos progridem e o GNN atinge a sua melhor taxa de sucesso (96,7%), ainda que com poucas recolhas. Em suma, o MARL (PPO/SAC) supera o evolutivo em todos os cenários quanto à tarefa efetiva; o GNN só se aproxima em espaço aberto, e a sua vantagem manifesta-se noutra dimensão — a escalabilidade (Secção 6.10).



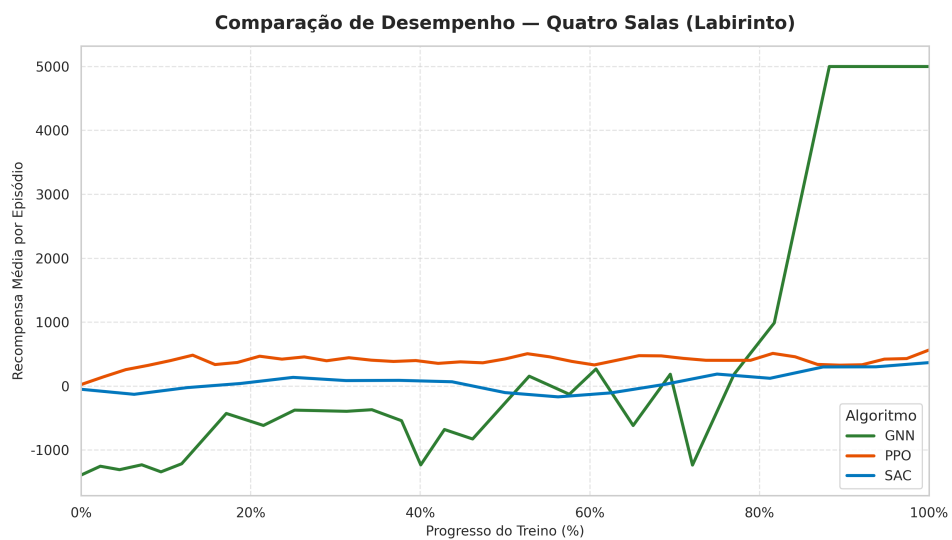
ao ninho (topo da arena). Exige que os agentes aprendam a explorar lateralmente. | Eixo X normalizado (0%→100% do treino de cada algoritmo) para comparação justa. GNN: fitness e

FIGURA 6.6. Curvas de aprendizagem no cenário Beco Sem Saída (Muro em U).



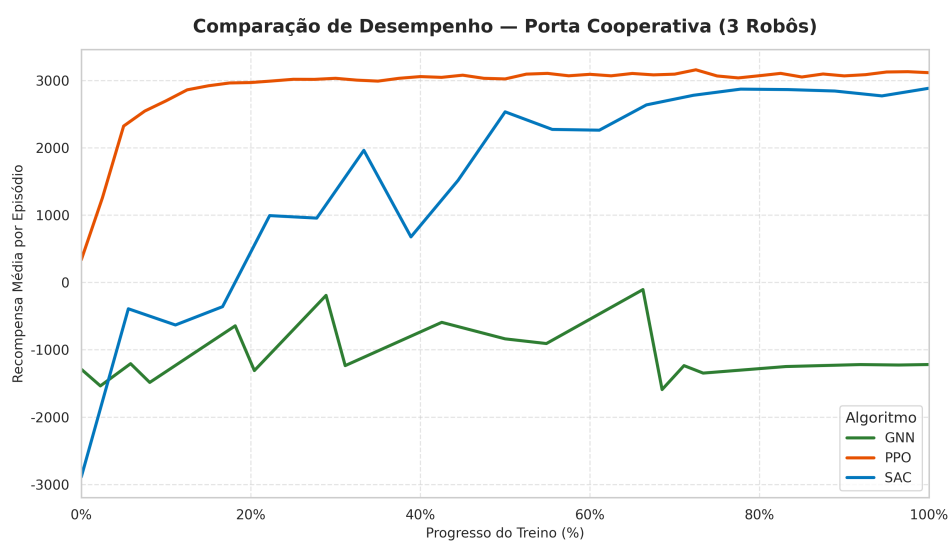
reita de 1,5m no centro. Testa gestão de congestionamento e fluxo cooperativo. | Eixo X normalizado (0%→100% do treino de cada algoritmo) para comparação justa. GNN: fitness evol

FIGURA 6.7. Curvas de aprendizagem no cenário Gargalo.



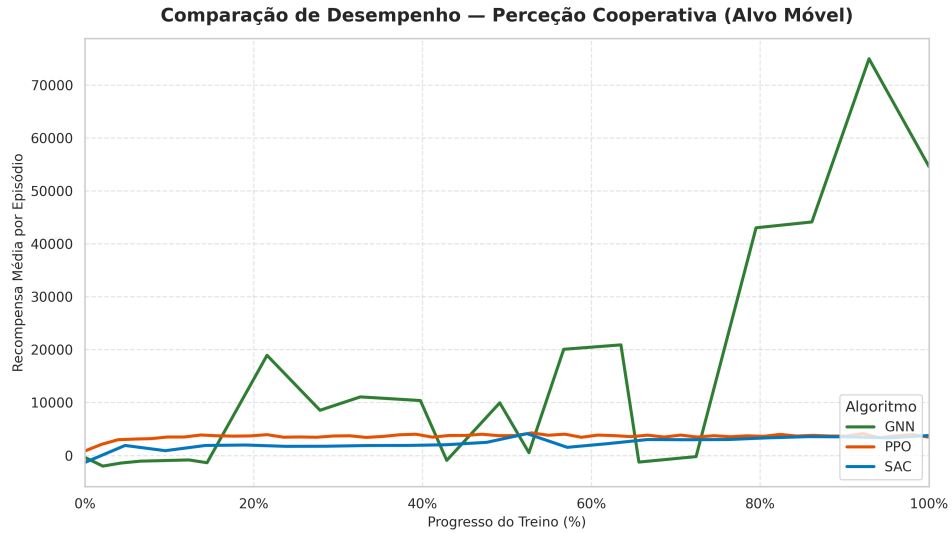
assagens específicas. Avalia navegação estruturada e memória espacial. | Eixo X normalizado (0%→100% do treino de cada algoritmo) para comparação justa. GNN: fitness evolutiva; P

FIGURA 6.8. Curvas de aprendizagem no cenário Quatro Salas (Labirinto).



is a empurrar em simultâneo. Testa coordenação emergente sem comunicação explícita. | Eixo X normalizado (0%→100% do treino de cada algoritmo) para comparação justa. GNN: fit

FIGURA 6.9. Curvas de aprendizagem no cenário Porta Cooperativa.

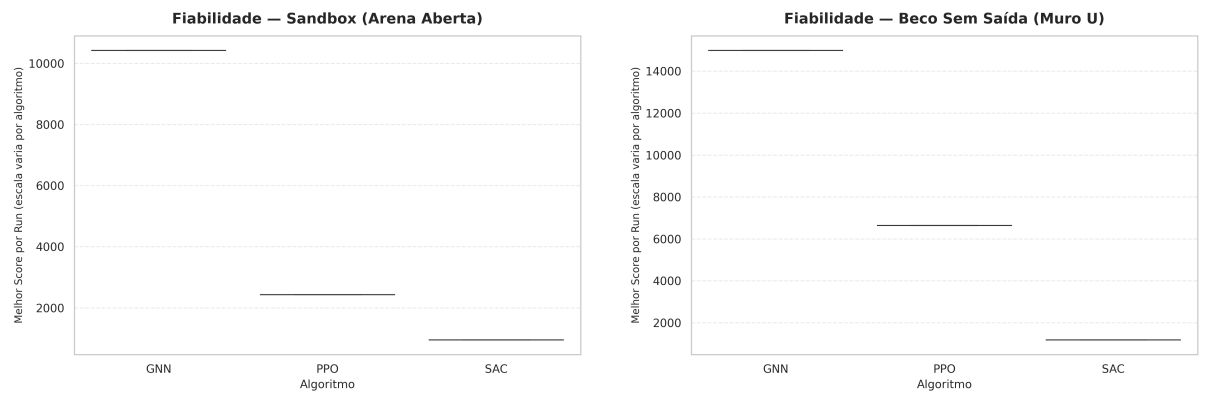


do rodeado por 3+ robôs a 360°. Avalia perseguição e encirclement cooperativo. | Eixo X normalizado (0%–100% do treino de cada algoritmo) para comparação justa. GNN: fitness evc

FIGURA 6.10. Curvas de aprendizagem no cenário Percepção Cooperativa (Alvo Móvel).

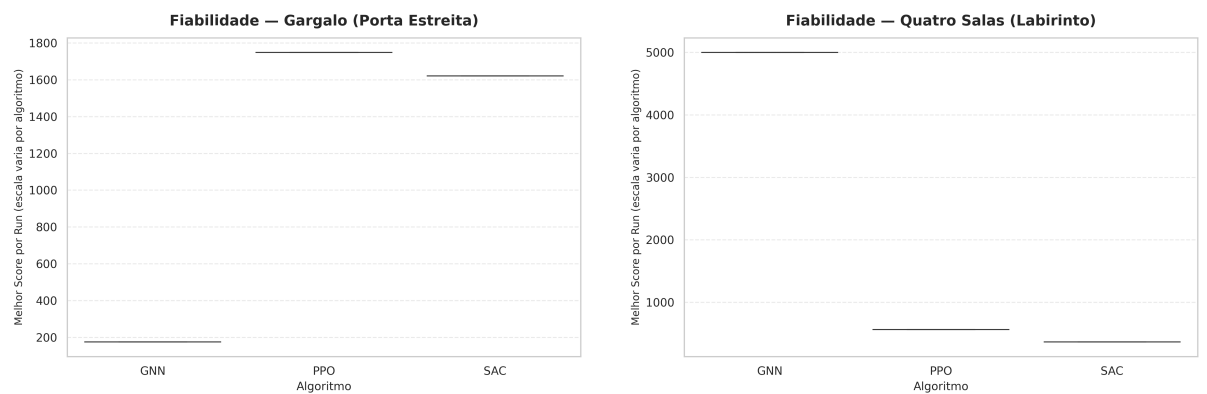
## 6.6. Fiabilidade e Variância entre *Runs* (Boxplots)

A análise da fiabilidade de treino requer múltiplas execuções independentes por configuração, de modo a distinguir o desempenho sistemático do ruído de inicialização. **Os resultados aqui apresentados correspondem a uma única execução por par algoritmo–cenário**; consequentemente, os diagramas de caixa das Figuras 6.11–6.13 degeneram num único ponto e não suportam, ainda, inferência sobre a variância entre *runs*. Apresentam-se como referência metodológica e espaço reservado: a bateria definitiva de **30 execuções por cenário e algoritmo** — exigida pelo protocolo experimental (Tabela 5.1) para validar a significância dos resultados — está prevista para a fase final de experimentação nos servidores de cálculo, e substituirá estas figuras pelas distribuições completas. A análise de significância estatística que se segue (Secção 6.13) é, por isso, conduzida sobre a distribuição dos **episódios de avaliação** (30 por configuração), que é a base estatística sólida atualmente disponível.



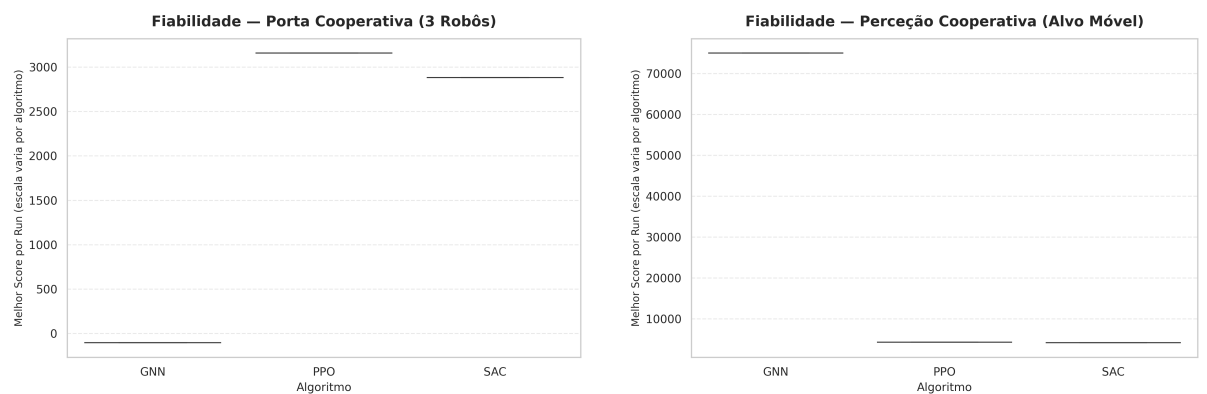
5. Avalia a capacidade base de forrageamento e orientação. | ATENCAO: GNN usa fitness evolutiva; PPO/SAC usam recompensa episódica (escalas diferentes). | 7e que os agentes aprendam a explorar lateralmente. | ATENCAO: GNN usa fitness evolutiva; PPO/SAC usam recompensa episódica (escalas diferentes).

FIGURA 6.11. Distribuição dos melhores scores por *run*: Sandbox (esq.) e Muro U (dir.).



1 gestão de congestionamento e fluxo cooperativo. | ATENCAO: GNN usa fitness evolutiva; PPO/SAC usam recompensa episódica (escalas diferentes). | 10 navegação estruturada e memória espacial. | ATENCAO: GNN usa fitness evolutiva; PPO/SAC usam recompensa episódica (escalas diferentes).

FIGURA 6.12. Distribuição dos melhores scores por *run*: Gargalo (esq.) e Quatro Salas (dir.).



11 Testa coordenação emergente sem comunicação explícita. | ATENCAO: GNN usa fitness evolutiva; PPO/SAC usam recompensa episódica (escalas diferentes). | 10 Avalia perseguição e encirclement cooperativo. | ATENCAO: GNN usa fitness evolutiva; PPO/SAC usam recompensa episódica (escalas diferentes).

FIGURA 6.13. Distribuição dos melhores scores por *run*: Porta Cooperativa (esq.) e Percepção Cooperativa (dir.).

## 6.7. Visão Global por Algoritmo

Consolidando o desempenho ao longo dos seis cenários (Figuras 6.14–6.16), emergem três perfis bem definidos. O **SAC** é o controlador mais *robusto à dificuldade do mapa*: é o único que não colapsa em nenhum cenário, mantendo 100% de sucesso mesmo no Muro U. O **PPO** é o mais *eficiente onde converge* — lidera em magnitude de recolhas no Sandbox e na Porta Cooperativa —, mas é *frágil* a um modo de falha específico: o *reward hacking* no Muro U, onde o bônus de exploração subverte o objetivo. O **GNN evolutivo** é o que mais cedo *colapsa*: degrada-se à medida que o cenário exige navegação por gargalos, falhando os três labirintos. A Figura 6.17 resume a comparação por barras; reitera-se que esse gráfico justapõe *fitness* (GNN) e recompensa (PPO/SAC) no mesmo eixo, pelo que é apresentado não como ranking direto mas como ilustração do próprio problema das escalas mistas que motiva o uso das métricas de tarefa.

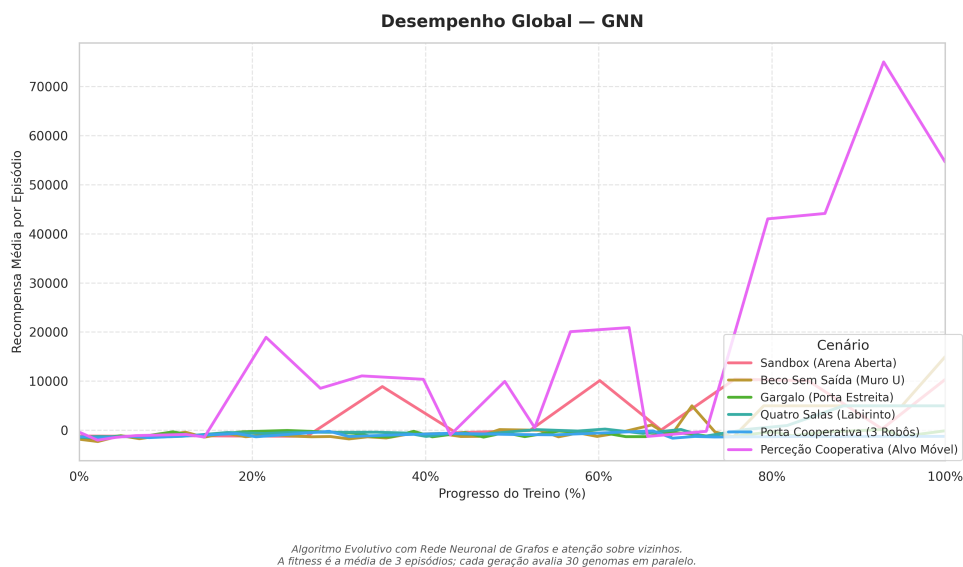


FIGURA 6.14. Desempenho do GNN (evolutivo) nos seis cenários.

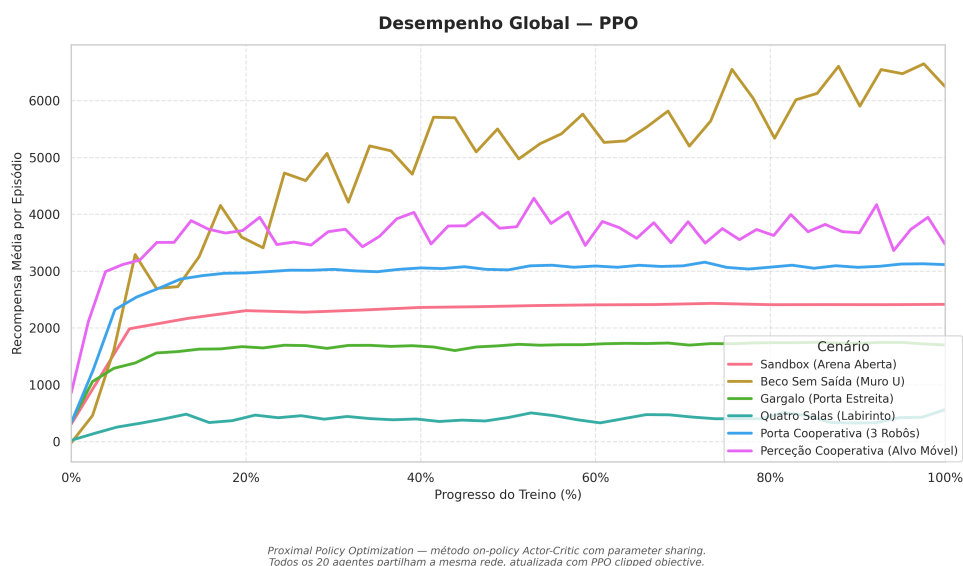


FIGURA 6.15. Desempenho do PPO nos seis cenários.

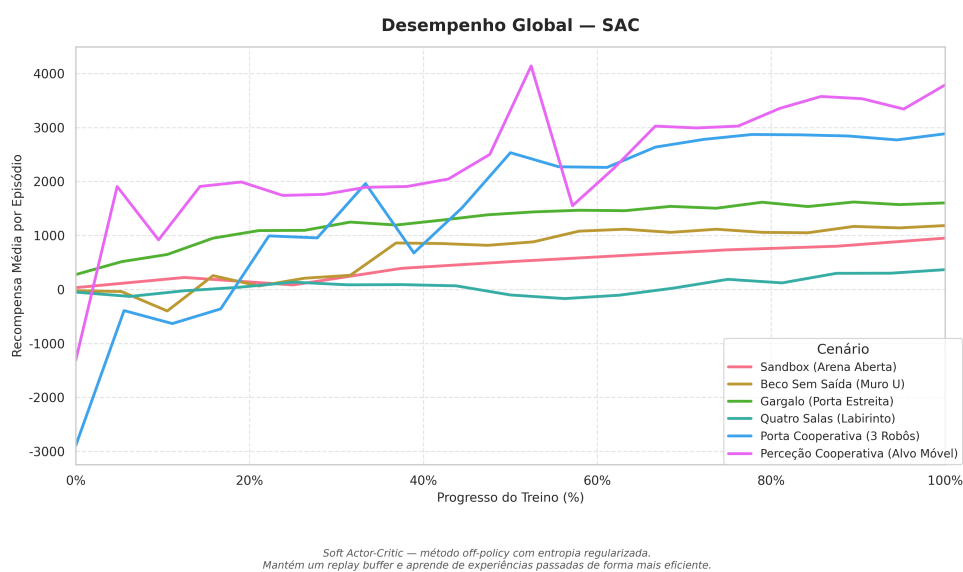


FIGURA 6.16. Desempenho do SAC nos seis cenários.

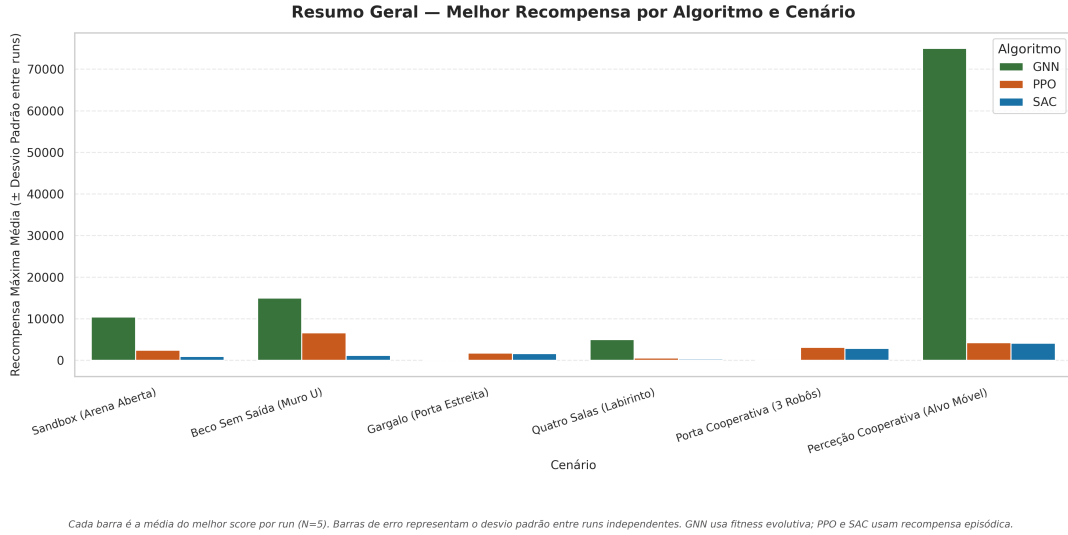


FIGURA 6.17. Resumo geral: melhor score médio por algoritmo e cenário (barras de erro = desvio padrão entre *runs*).

## 6.8. Avaliação Determinística (Tarefa Pura)

A Tabela 6.2 reúne a métrica mais honesta do desempenho real na missão: avaliação em modo determinístico (sem ruído de exploração), com 30 episódios emparelhados por configuração. Os valores fundamentam a discussão das secções anteriores — o domínio do SAC em navegação, o colapso isolado do PPO no Muro U e a falha do GNN nos cenários de gargalo.

TABELA 6.2. Avaliação determinística (30 episódios emparelhados): taxa de sucesso ( $P_{task}$ ) e recolhas médias por episódio.

| Cenário           | GNN     |          | PPO     |          | SAC     |          |
|-------------------|---------|----------|---------|----------|---------|----------|
|                   | Sucesso | Recolhas | Sucesso | Recolhas | Sucesso | Recolhas |
| Sandbox           | 66,7%   | 0,77     | 100%    | 73,30    | 100%    | 25,23    |
| Muro U            | 43,3%   | 0,43     | 0%      | 0,00     | 100%    | 20,63    |
| Gargalo           | 0%      | 0,00     | 100%    | 41,03    | 100%    | 35,00    |
| Quatro Salas      | 0%      | 0,00     | 100%    | 11,50    | 100%    | 11,60    |
| Porta Cooperativa | 0%      | 0,00     | 100%    | 67,73    | 100%    | 60,50    |
| Percepção Coop.   | 96,7%   | 2,63     | 100%    | 17,87    | 100%    | 16,23    |

## 6.9. Análise de Cenários Complexos e Comportamento Emergente

Para além das métricas agregadas, a inspeção qualitativa no visualizador 3D (Figuras 6.18–6.19) revela os comportamentos coletivos que sustentam — ou comprometem — o desempenho de tarefa. No **Muro U** e no **Quatro Salas** (Figura 6.18), as políticas bem-sucedidas exibem contorno fluido das paredes, com os agentes a seguirem o gradiente geodésico em vez de colidirem frontalmente com os obstáculos — o efeito direto da

substituição do potencial euclidiano discutida na Secção 6.3. A dispersão inicial do *spawn* pelas salas opostas ao ninho mitiga o congestionamento nas passagens estreitas, que era a causa do colapso anterior no Quatro Salas.

Nos cenários cooperativos (Figura 6.19) emergem comportamentos coordenados não programados explicitamente: na **Porta Cooperativa**, observa-se a sincronização de pelo menos três agentes na zona de pressão para abrir a passagem, condição imposta pela mecânica do cenário; na **Percepção Cooperativa**, o enxame organiza-se em torno do alvo móvel, mantendo cobertura à medida que este se desloca. Estes padrões confirmam que os controladores de gradiente aprendem não só a navegar, mas a explorar a estrutura cooperativa da tarefa. (As capturas apresentadas são ilustrativas do comportamento típico observado em execução.)

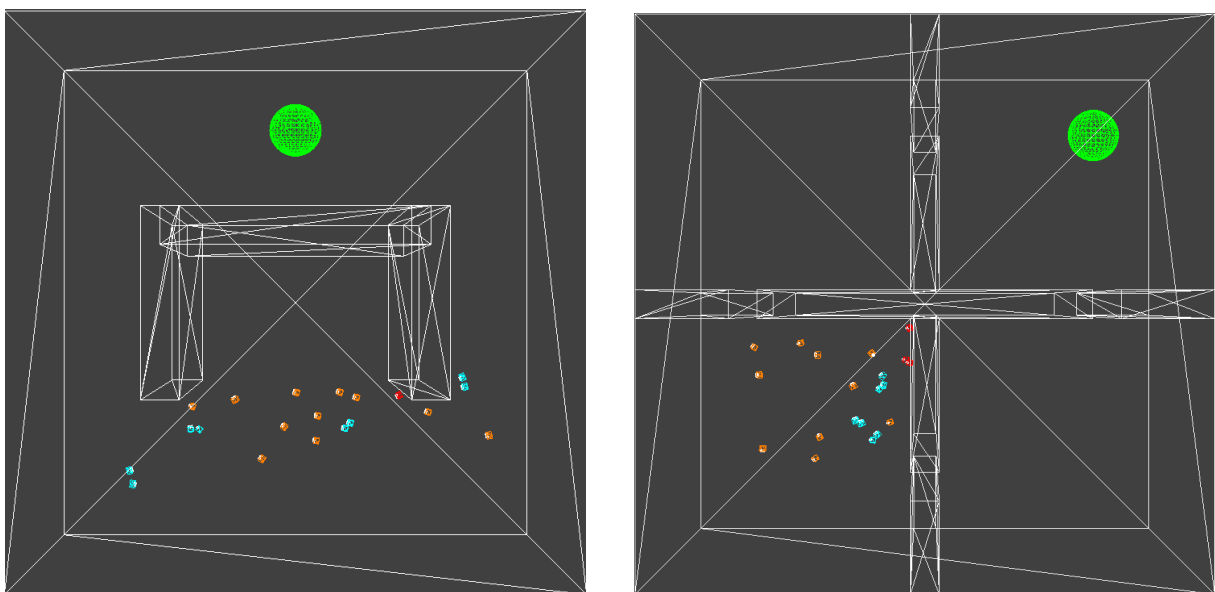


FIGURA 6.18. Comportamento emergente no visualizador 3D: contorno do obstáculo em U (esq.) e navegação no labirinto de Quatro Salas (dir.).

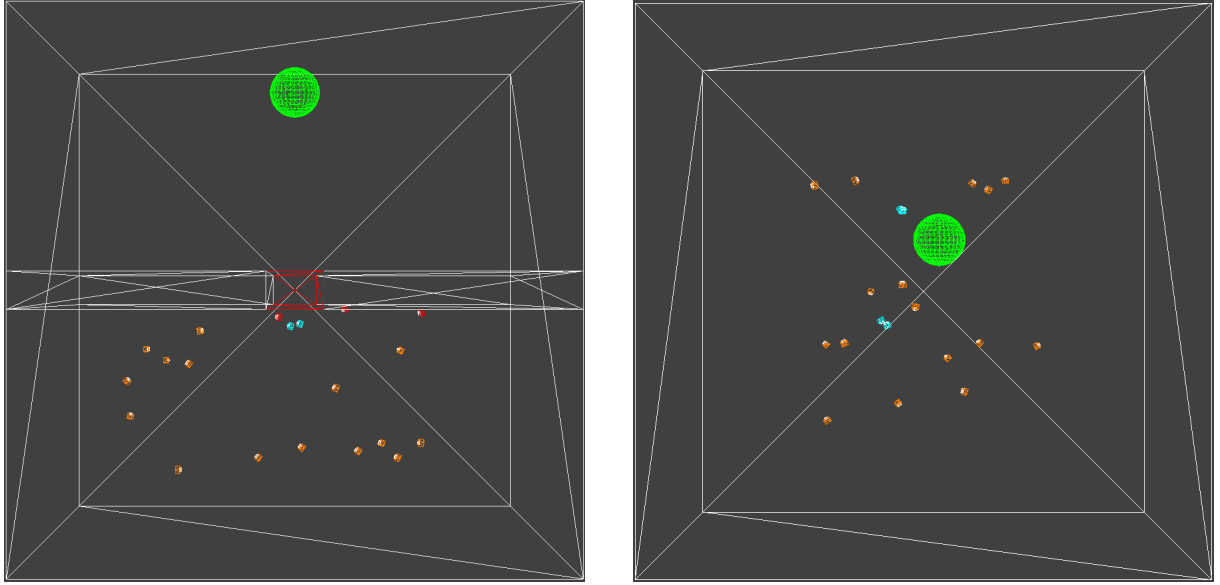


FIGURA 6.19. Comportamento cooperativo: abertura sincronizada da porta (esq.) e cerco a 360° do alvo móvel (dir.).

### 6.10. Análise de Escalabilidade (Zero-Shot Transfer)

Se em desempenho de tarefa o GNN evolutivo fica aquém dos métodos de gradiente, é na **escalabilidade** que reside o seu contributo distintivo — e a justificação arquitetural da abordagem por atenção sobre grafo. A Tabela 6.3 e a Figura 6.20 reportam o  $P_{task}$  ao transferir, *sem qualquer retreino*, a política aprendida com  $N = 20$  agentes para enxames de  $N \in \{10, 50, 100\}$ .

O resultado é inequívoco: o GNN é o **único** algoritmo capaz de *Zero-Shot Transfer* e, mais notavelmente, **escala positivamente** — o sucesso sobe de 15% ( $N = 10$ ) para 50% ( $N = 20$ ), 95% ( $N = 50$ ) e 100% ( $N = 100$ ), e as recolhas por agente mantêm-se estáveis ( $0,015 \rightarrow 0,034$ ), ou seja, sem degradação per capita com o aumento da densidade. As políticas MLP do PPO e do SAC, com dimensão de entrada fixa ( $\mathbb{R}^{111}$ , calibrada para  $N = 20$ ), são por construção incompatíveis com outras dimensões de exame e marcadas como N/A. Este contraste corrobora diretamente a hipótese de que o mecanismo de atenção sobre vizinhança confere invariância à cardinalidade do exame, sendo a propriedade que habilita o controlo escalável.

TABELA 6.3. Escalabilidade Zero-Shot:  $P_{task}$  ao transferir a política treinada com  $N = 20$  para outras dimensões de enxame, sem retreino.

| Algoritmo       | $N = 10$         | $N = 20$ (treino) | $N = 50$         | $N = 100$        |
|-----------------|------------------|-------------------|------------------|------------------|
| GNN (Evolutivo) | 15,0%            | 50,0%             | 95,0%            | 100%             |
| PPO             | N/A <sup>†</sup> | 100%              | N/A <sup>†</sup> | N/A <sup>†</sup> |
| SAC             | N/A <sup>†</sup> | 100%              | N/A <sup>†</sup> | N/A <sup>†</sup> |

<sup>†</sup> A política MLP do PPO/SAC tem dimensão de entrada fixa ( $\mathbb{R}^{111}$ , para  $N = 20$ ), sendo incompatível com outras dimensões de enxame. Só a arquitetura de atenção sobre grafo (controlador evolutivo) suporta a transferência *Zero-Shot*.

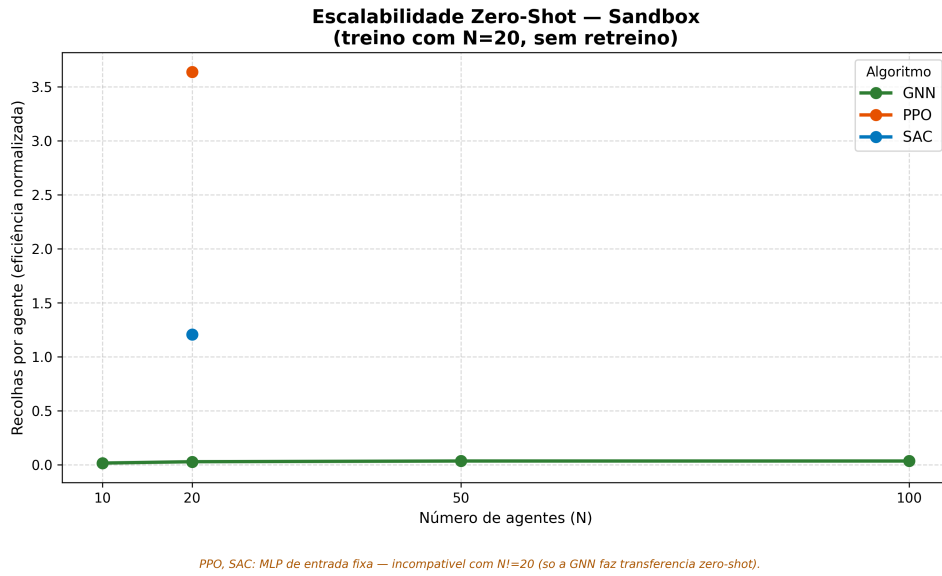


FIGURA 6.20. Degradação (ou estabilidade) do desempenho com o aumento do número de agentes, sem retreino.

### 6.11. Resiliência e Robustez a Falhas de Agentes

A métrica  $R_{robust}$  avalia a resiliência do enxame perante a perda súbita de 10% dos agentes a meio do episódio (os agentes afetados ficam inertes, deixando de agir e de contar para os requisitos cooperativos). A Figura 6.21 compara as recolhas antes e depois da falha, em avaliação emparelhada com a linha de base sem falhas.

O resultado evidencia uma resiliência elevada dos métodos de gradiente: PPO e SAC retêm aproximadamente **94–99% das recolhas** em todos os cenários, incluindo a Porta Cooperativa ( $\approx 94\%$  de retenção), notável por exigir três agentes simultâneos para abrir a passagem — o enxame redistribui-se e mantém a função apesar da baixa. O GNN evolutivo, onde tem desempenho, retém entre 69% e 96%, mais sensível à perda mas sem colapso total. A ausência de degradação catastrófica sugere que o *parameter sharing* e a observação local conferem ao enxame uma redundância funcional implícita: nenhum agente é insubstituível, pelo que a falha parcial é absorvida pela reorganização do coletivo.

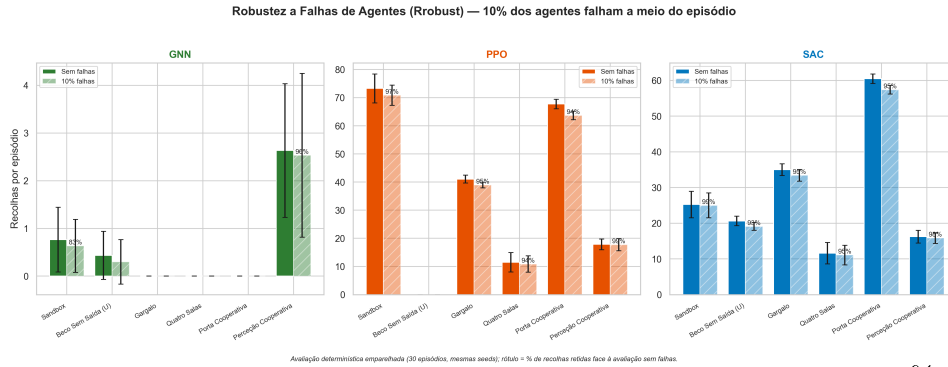


FIGURA 6.21. Resiliência do enxame perante a perda súbita de 10% dos agentes a meio do episódio ( $R_{robust}$ ).

## 6.12. Desempenho Computacional do Simulador

A viabilidade da campanha experimental depende do *throughput* do simulador. Em execução de referência *single-thread* (uma arena,  $N = 20$  agentes), o ambiente sustenta cerca de **139 passos de simulação por segundo** — equivalente a aproximadamente 2770 atualizações de agente por segundo —, medidos na máquina de desenvolvimento sobre 3000 passos com ações aleatórias (Tabela 6.4). A esta taxa, um episódio completo (500 passos) demora  $\approx 3,6$  s.

O treino dos métodos de gradiente escapa a este limite por **vetorização**: PPO e SAC executam 16 clones independentes da arena em processos paralelos (**SubprocVecEnv**), recolhendo experiência concorrentemente nos vários núcleos dos servidores de cálculo. O controlador evolutivo, por sua vez, avalia os  $K$  genomas da população em paralelo. Em ambos os casos, a aceleração efetiva aproxima-se do número de núcleos disponíveis, ficando aquém do limite teórico devido ao custo de comunicação entre processos e à serialização das observações — a quantificação exata deste *speedup* constitui trabalho de instrumentação a realizar na fase final.

TABELA 6.4. Desempenho computacional do simulador (*throughput* de referência,  $N = 20$ ).

| Configuração                                          | Throughput                                           |
|-------------------------------------------------------|------------------------------------------------------|
| Single-thread (1 arena)                               | $\approx 139$ passos/s ( $\approx 2770$ ag.-passo/s) |
| Episódio completo (500 passos)                        | $\approx 3,6$ s                                      |
| Treino vetorizado ( <b>SubprocVecEnv</b> , 16 arenas) | paralelização $\approx n.^o$ de núcleos              |

## 6.13. Discussão Global e Validação da Hipótese

A hipótese central desta investigação postula que a *inteligência adaptativa* (MARL) supera a *robustez estática* bio-inspirada em cenários de *stress* dinâmico. Os resultados obtidos exigem uma resposta diferenciada, e não um veredicto único.

Desempenho de tarefa. Em cumprimento efetivo da missão, os métodos de gradiente dominam de forma clara. O SAC resolve os seis cenários; o PPO resolve cinco, falhando

apenas o Muro U por *reward hacking*. O controlador evolutivo (GNN) fica sistematicamente aquém, colapsando nos cenários de gargalo. Para quantificar estas diferenças, aplicou-se o teste não-paramétrico de Mann-Whitney sobre as recolhas por episódio (métrica comparável entre paradigmas), complementado pelo  $t$  de Welch; a Tabela 6.5 reporta os resultados.

TABELA 6.5. Significância estatística das diferenças em recolhas por episódio (Mann-Whitney U; 30 episódios emparelhados;  $\alpha = 0,05$ ).

| Cenário               | Par        | média A | média B | $p$        | Significativo |
|-----------------------|------------|---------|---------|------------|---------------|
| Sandbox               | GNN vs PPO | 0,77    | 73,30   | $< 0,0001$ | Sim           |
| Sandbox               | GNN vs SAC | 0,77    | 25,23   | $< 0,0001$ | Sim           |
| Sandbox               | PPO vs SAC | 73,30   | 25,23   | $< 0,0001$ | Sim           |
| Beco Sem Saída (U)    | GNN vs PPO | 0,43    | 0,00    | 0,0003     | Sim           |
| Beco Sem Saída (U)    | GNN vs SAC | 0,43    | 20,63   | $< 0,0001$ | Sim           |
| Beco Sem Saída (U)    | PPO vs SAC | 0,00    | 20,63   | $< 0,0001$ | Sim           |
| Gargalo               | GNN vs PPO | 0,00    | 41,03   | $< 0,0001$ | Sim           |
| Gargalo               | GNN vs SAC | 0,00    | 35,00   | $< 0,0001$ | Sim           |
| Gargalo               | PPO vs SAC | 41,03   | 35,00   | $< 0,0001$ | Sim           |
| Quatro Salas          | GNN vs PPO | 0,00    | 11,50   | $< 0,0001$ | Sim           |
| Quatro Salas          | GNN vs SAC | 0,00    | 11,60   | $< 0,0001$ | Sim           |
| Quatro Salas          | PPO vs SAC | 11,50   | 11,60   | 0,5312     | não           |
| Porta Cooperativa     | GNN vs PPO | 0,00    | 67,73   | $< 0,0001$ | Sim           |
| Porta Cooperativa     | GNN vs SAC | 0,00    | 60,50   | $< 0,0001$ | Sim           |
| Porta Cooperativa     | PPO vs SAC | 67,73   | 60,50   | $< 0,0001$ | Sim           |
| Percepção Cooperativa | GNN vs PPO | 2,63    | 17,87   | $< 0,0001$ | Sim           |
| Percepção Cooperativa | GNN vs SAC | 2,63    | 16,23   | $< 0,0001$ | Sim           |
| Percepção Cooperativa | PPO vs SAC | 17,87   | 16,23   | $< 0,0001$ | Sim           |

Das 18 comparações par-a-par, 17 são estatisticamente significativas ( $p < 0,001$ ); a única exceção é PPO vs. SAC no Quatro Salas (11,50 vs. 11,60 recolhas;  $p = 0,53$ ), onde os dois métodos de gradiente são indistinguíveis. A superioridade do MARL sobre o controlador evolutivo na tarefa é, portanto, robusta e não atribuível ao acaso.

Anatomia do colapso evolutivo. A inferioridade sistemática do controlador evolutivo nos cenários de gargalo não decorre da arquitetura da rede — a mesma representação de grafo com atenção sustenta a vantagem de escalabilidade discutida adiante — mas da *interação entre o método de otimização e a estrutura da recompensa*. A neuroevolução otimiza os pesos a partir de um único escalar de *fitness* por episódio, sem qualquer atribuição de crédito temporal: ao contrário dos métodos de gradiente, a evolução não dispõe de informação sobre *qual* ação, em *que* instante, contribuiu para o resultado; observa apenas

o desfecho agregado. Esta cegueira torna-se crítica quando a recompensa de tarefa é esparsa. Recolher alimento num labirinto exige uma sequência longa e coordenada de ações — contornar o obstáculo, alcançar o ninho e regressar — cuja probabilidade de emergir por mutação aleatória dos pesos é praticamente nula. Em consequência, a quase totalidade dos genomas da população recolhe zero unidades de alimento, e a respetiva *fitness* reduz-se ao termo de *reward shaping*; saturado pela função tanh, este converge para um valor comum a toda a população. O conjunto instala-se assim num *planalto de fitness* onde a pressão seletiva se anula: não havendo forma de distinguir os genomas promissores dos restantes, a evolução perde direção e deambula pelo espaço de parâmetros. É esta a mecânica subjacente ao fenómeno de *fitness exploitation* já assinalado — a métrica de treino estabiliza num ótimo do *shaping* (por exemplo, encostar ao muro mais próximo do ninho) que não corresponde ao cumprimento efetivo da missão.

Variância entre execuções e profundidade de treino. A mesma natureza estocástica explica a elevada variância do controlador evolutivo entre execuções independentes, ausente nos métodos de gradiente. Sem um gradiente que conduza todas as execuções para um mesmo ótimo, cada *run* percorre uma trajetória aleatória distinta pela paisagem de *fitness*, determinada pela inicialização dos pesos e pela sequência de mutações. Nos cenários abertos, onde múltiplas políticas medianas são adequadas, a generalidade das execuções converge para soluções funcionais, ainda que de qualidade dispersa; nos cenários de gargalo, onde o conjunto de soluções viáveis forma um vale estreito na paisagem, encontrá-lo depende sobretudo do número de gerações exploradas — isto é, da *profundidade* de treino concedida a cada execução. Observou-se que execuções com maior orçamento temporal por *run* chegam ocasionalmente a resolver o Muro U e a Porta Cooperativa, ao passo que execuções mais curtas, mesmo quando repetidas em maior número, raramente o conseguem. Esta sensibilidade indica que a limitação é de *eficiência de amostragem* do otimizador, e não de capacidade representacional da rede — distinção que a arquitetura assimétrica adotada (Secção 7.2) impede de quantificar de forma isolada e que motiva diretamente a integração futura do módulo de atenção numa política treinável por gradiente.

Escalabilidade. O quadro inverte-se quando se considera a transferência *Zero-Shot* (Secção 6.10). Aqui, o controlador evolutivo com atenção sobre grafo é o *único* viável, escalando de  $N = 10$  a  $N = 100$  sem retreino e sem degradação per capita, enquanto as políticas MLP do PPO/SAC são estruturalmente incompatíveis com  $N \neq 20$ . A adaptabilidade arquitetural que a hipótese atribui ao MARL manifesta-se, assim, não no algoritmo de otimização, mas na *representação* (grafo com atenção vs. vetor de dimensão fixa).

Robustez. Perante falhas de 10% dos agentes, todos os paradigmas exibem resiliência elevada (Secção 6.11), sem o colapso que a hipótese poderia antecipar para o controlador estático — o *parameter sharing* confere redundância funcional transversal.

Veredicto. A hipótese confirma-se *parcialmente* e de forma esclarecedora: não existe um paradigma universalmente superior, mas sim um **trade-off central**. O MARL

(SAC/PPO) vence onde a métrica é o cumprimento da tarefa num exame de dimensão fixa; a arquitetura de grafo (GNN evolutivo) vence onde a métrica é a escalabilidade a dimensões não vistas. O resultado mais valioso para a engenharia de exames não é eleger um vencedor, mas identificar este ponto de inflexão — a escolha do paradigma deve ser ditada pelo eixo de exigência dominante da aplicação. Importa por fim a ressalva metodológica (Secção 6.6): estas conclusões assentam numa única execução de treino por configuração; a bateria de 30 execuções independentes, prevista para a fase final, consolidará a estimativa da variância entre *runs* sem que se anteveja alteração do quadro qualitativo aqui estabelecido.

[ Página intencionalmente deixada em branco. ]

## Conclusões e Trabalhos Futuros

### 7.1. Conclusões

Esta dissertação incidiu sobre um dos desafios mais prementes na robótica contemporânea: a definição de paradigmas de controlo que garantam a operabilidade de enxames em ambientes de alta incerteza. Identificou-se uma lacuna crítica na literatura — o isolamento metodológico entre as comunidades de *Multi-Agent Reinforcement Learning* (MARL) e de Otimização Bio-inspirada — e respondeu-se a essa lacuna com um **benchmarking rigoroso e imparcial**: desenvolveu-se um simulador de alta fidelidade, implementou-se o *framework* RS2C (com PPO e SAC) e um controlador neuroevolutivo assente numa rede de grafos com atenção, e compararam-se os três algoritmos nos mesmos seis cenários, sob avaliação determinística e tratamento estatístico.

Os resultados (Capítulo 6) não elegem um vencedor universal, mas revelam um **trade-off central**. No cumprimento efetivo da tarefa, os métodos de gradiente dominam: o SAC resolve os seis cenários e o PPO cinco (falhando apenas o Muro U, por *reward hacking*), ao passo que o controlador evolutivo colapsa nos cenários de gargalo — um caso de *fitness exploitation* em que a *fitness* de treino sobe sem se traduzir em recolhas. Em contrapartida, na transferência **Zero-Shot** o quadro inverte-se: só a arquitetura de atenção sobre grafo escala de  $N = 10$  a  $N = 100$  sem retreino e sem degradação per capita, enquanto as políticas MLP de dimensão fixa do PPO/SAC são estruturalmente incompatíveis com  $N \neq 20$ . A adaptabilidade que a hipótese atribuíra ao MARL manifesta-se, afinal, não no algoritmo de otimização, mas na *representação* (grafo com atenção vs. vetor de dimensão fixa). Perante falhas de 10% dos agentes, todos os paradigmas exibem resiliência elevada, conferida pela partilha de parâmetros e pela observação local.

Confirma-se, assim, *parcialmente* a hipótese: a escolha do paradigma deve ser ditada pelo eixo de exigência dominante da aplicação — desempenho de tarefa num exame de dimensão fixa (MARL) versus escalabilidade a dimensões não vistas (grafo com atenção). Importa a ressalva metodológica de que estas conclusões assentam, por ora, numa única execução de treino por configuração; a bateria de 30 execuções independentes (Secção 7.4) consolidará a estimativa de variância, não se antevendo alteração do quadro qualitativo aqui estabelecido.

### 7.2. Limitações do Estudo

Em prol da transparência científica, identificam-se as seguintes limitações da implementação atual:

- **Arquitetura assimétrica entre controladores:** os *baselines* MARL (PPO e SAC) empregam uma política totalmente ligada (MLP) sobre a observação de vizinhança densa, enquanto o mecanismo de atenção sobre grafo é instanciado apenas no controlador neuroevolutivo. Esta opção — ditada pelo uso das implementações de referência da *Stable-Baselines3* — implica que a comparação entre paradigmas combina dois fatores (algoritmo de otimização e arquitetura da rede). A integração do módulo de atenção numa política treinável por gradiente (*custom policy*) é a extensão prioritária de trabalho futuro, permitindo isolar o efeito do otimizador.
- **Escalabilidade restrita ao controlador de grafo:** como consequência do ponto anterior, o *Zero-Shot Transfer* para  $N \neq 20$  só é diretamente avaliável no controlador com atenção; a MLP de entrada fixa do PPO/SAC é, por construção, incompatível com outras dimensões de enxame.
- **Crítico não-centralizado:** a fase de treino utiliza um crítico que parte da mesma observação local do ator (*parameter sharing*), não explorando o estado global. Trata-se de execução e treino descentralizados, e não de uma arquitetura CTDE com crítico centralizado em sentido estrito.
- ***Sim-to-real* não validado:** a totalidade dos resultados é obtida em simulação; a transferência para plataformas físicas (*Deployment Gap*) permanece por validar empiricamente.

### 7.3. Contributos Esperados

Com a execução deste plano de investigação, preveem-se os seguintes contributos para o avanço do conhecimento na área de Sistemas Multiagente e Robótica de Enxame:

- **Framework de Benchmarking Quantitativo:** A disponibilização de uma metodologia de teste padronizada que permita comparar algoritmos de naturezas distintas (aprendizagem vs. otimização) sob as mesmas métricas de performance, escalabilidade e robustez.
- **Validação da Adaptabilidade Zero-Shot:** A demonstração empírica de como arquiteturas baseadas em grafos permitem que políticas aprendidas via MARL escalem de  $N = 10$  para  $N = 100$  agentes sem degradação crítica de performance, respondendo aos desafios de escalabilidade propostos por Chen et al. (2024).
- **Identificação do Ponto de Inflexão Tecnológica:** O fornecimento de dados que permitam a investigadores e engenheiros determinar o limiar de complexidade ambiental a partir do qual o investimento computacional no treino de modelos MARL se traduz numa vantagem operacional real face às heurísticas evolutivas.
- **Implementação de Referência:** A criação de um repositório de código contendo o simulador e as implementações dos algoritmos (o *framework* RS2C com PPO/SAC e o controlador neuroevolutivo AE), servindo de base para futuros estudos de hibridização no ISCTE.

## 7.4. Trabalhos Futuros

A partir das limitações identificadas (Secção 7.2) e do estado atual da experimentação, delineiam-se as seguintes linhas de trabalho prioritárias:

- (1) **Consolidação estatística (30 runs):** Repetir o treino de cada par algoritmo–cenário em 30 execuções independentes com sementes distintas, substituindo os diagramas de caixa degenerados (Secção 6.6) por distribuições completas e robustecendo a estimativa da variância entre *runs*. É a extensão de maior prioridade, já em curso nos servidores de cálculo.
- (2) **Política de gradiente com atenção (*custom policy*):** Integrar o mecanismo de atenção sobre grafo numa política treinável por gradiente (PPO/SAC com extrator de *features* personalizado na *Stable-Baselines3*), de modo a *isolar* o efeito do algoritmo de otimização do efeito da arquitetura — removendo a assimetria MLP vs. grafo que atualmente confunde a comparação (Secção 7.2).
- (3) **Novos cenários e tarefas:** Implementar tarefas de cobertura/vigilância de área (para além do *foraging* e do cerco de alvos móveis) e cenários com percursos alternativos, ampliando o espaço de avaliação de comportamentos coletivos.
- (4) **Validação *Sim-to-Real*:** Transferir as políticas para plataformas físicas (e.g. e-puck, Khepera IV) ou para um simulador de física de maior fidelidade, quantificando empiricamente o *Deployment Gap*.
- (5) **Hibridização de paradigmas:** Explorar arquiteturas que combinem a escalabilidade do controlador evolutivo com a eficiência de tarefa dos métodos de gradiente (e.g. inicialização evolutiva seguida de afinação por RL).

[ Página intencionalmente deixada em branco. ]

## Referências

- [1] M. Hüttenrauch, A. Šošić e G. Neumann, «Deep Reinforcement Learning for Swarm Systems,» *Journal of Machine Learning Research*, vol. 20, n.º 54, pp. 1–31, 2019.
- [2] M. Tegmark, *Life 3.0: Being Human in the Age of Artificial Intelligence*. Alfred A. Knopf, 2017.
- [3] M. Dorigo, «Optimization, Learning and Natural Algorithms,» tese de doutoramento, Politecnico di Milano, 1992.
- [4] J. Kennedy e R. Eberhart, «Particle swarm optimization,» em *Proceedings of ICNN'95 - International Conference on Neural Networks*, IEEE, vol. 4, 1995, pp. 1942–1948.
- [5] C. C. Ekechi, T. Elfouly, A. Alouani e T. Khattab, «A Survey on UAV Control with Multi-Agent Reinforcement Learning,» *Drones*, vol. 9, n.º 7, p. 484, 2025.
- [6] J. He et al., «Towards Self-Adaptive Resilient Swarms Using Multi-Agent Reinforcement Learning,» em *Proceedings of the International Conference on Agents and Artificial Intelligence (ICAART)*, SciTePress, 2024.
- [7] A. Y. Majid, S. Saaybi, V. Francois-Lavet, R. V. Prasad e C. Verhoeven, «Deep Reinforcement Learning Versus Evolution Strategies: A Comparative Survey,» *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, n.º 9, pp. 11 939–11 957, 2024.
- [8] W. M. Hassen e S. H. Amin, «Dynamic Window Approach for Mobile Robot Path Planning based on Hybrid Swarm Algorithms,» *Journal of Information Hiding and Multimedia Signal Processing*, vol. 16, n.º 2, 2025.
- [9] T. Salimans, J. Ho, X. Chen, S. Sidor e I. Sutskever, «Evolution Strategies as a Scalable Alternative to Reinforcement Learning,» *arXiv preprint arXiv:1703.03864*, 2017.
- [10] S. Somvanshi, M. M. Islam, S. A. Javed, G. Chhetri, K. S. Islam, T. I. Chowdhury, S. B. B. Pollock, A. Dutta e S. Das, «A Review on Influx of Bio-Inspired Algorithms: Critique and Improvement Needs,» *arXiv preprint arXiv:2506.04238*, 2025.
- [11] R. S. Sutton e A. G. Barto, *Reinforcement Learning: An Introduction*, 2ª ed. MIT Press, 2018.
- [12] D. Huh e P. Mohapatra, «Multi-agent Reinforcement Learning: A Comprehensive Survey,» *arXiv preprint arXiv:2312.10256*, 2023.
- [13] M. Bettini, A. Prorok e V. Moens, «BenchMARL: Benchmarking Multi-Agent Reinforcement Learning,» *Journal of Machine Learning Research*, vol. 25, n.º 161, 2024.
- [14] P. Li, J. Hao, H. Tang, Y. Zheng e X. Fu, «RACE: Improve Multi-Agent Reinforcement Learning with Representation Asymmetry and Collaborative Evolution,»

- em *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023, pp. 19 490–19 503.
- [15] M. Kegeleirs e M. Birattari, «Towards applied swarm robotics: current limitations and enablers,» *Frontiers in Robotics and AI*, vol. 12, 2025. DOI: [10.3389/frobt.2025.1607978](https://doi.org/10.3389/frobt.2025.1607978)
  - [16] X. Chen, N. NaderiAlizadeh, A. Ribeiro e S. Saeedi Bidokhti, «Transferable Graphical MARL for Real-Time Estimation in Dynamic Wireless Networks,» *arXiv preprint arXiv:2404.03227*, 2024.
  - [17] T. Schmickl et al., «Scaling Swarm Coordination with GNNs—How Far Can We Go?» *AI*, vol. 6, n.<sup>o</sup> 11, p. 282, 2025. DOI: [10.3390/ai6110282](https://doi.org/10.3390/ai6110282)
  - [18] D. Heimann et al., «Runtime Verification-Based Safe MARL for Optimized Safety Policy Generation,» *MDPI Big Data and Cognitive Computing*, vol. 8, n.<sup>o</sup> 5, 2024.
  - [19] F. A. Oliehoek e C. Amato, *A Concise Introduction to Decentralized POMDPs*. Springer, 2016.
  - [20] J. Schulman, F. Wolski, P. Dhariwal, A. Radford e O. Klimov, «Proximal Policy Optimization Algorithms,» *arXiv preprint arXiv:1707.06347*, 2017.
  - [21] J. Schulman, P. Moritz, S. Levine, M. I. Jordan e P. Abbeel, «High-Dimensional Continuous Control Using Generalized Advantage Estimation,» em *International Conference on Learning Representations (ICLR)*, 2016.
  - [22] T. Haarnoja, A. Zhou, P. Abbeel e S. Levine, «Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor,» em *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 1861–1870.
  - [23] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel e I. Mordatch, «Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments,» em *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
  - [24] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser e I. Polosukhin, «Attention is All You Need,» em *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
  - [25] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò e Y. Bengio, «Graph Attention Networks,» em *International Conference on Learning Representations (ICLR)*, 2018.
  - [26] Y. Wang, T. Duhan, J. Li e G. Sartoretti, «LNS2+RL: Combining Multi-Agent Reinforcement Learning with Large Neighborhood Search in Multi-Agent Path Finding,» *arXiv preprint arXiv:2405.17794*, 2025.
  - [27] W. Xiao, L. Yuan, T. Ran, L. He, J. Zhang e J. Cui, «CSPER: Curiosity-driven Safety-Prioritized Experience Replay for Mapless Navigation using Deep Reinforcement Learning,» *IEEE Transactions on Cognitive and Developmental Systems*, 2026.

- [28] L. Roveda, «Hybrid Reinforcement Learning and Evolutionary Algorithms for Adaptive Robotic Manipulation,» *Robotics and Computer-Integrated Manufacturing*, 2026.
- [29] O. Yigit, S. E. Ceylan, S. S. Palas, A. Isik, E. Bekar, B. Vatansever e Y. Oniz, «Multi Agent Reinforcement Learning Based Swarm Navigation,» em *2026 6th International Congress on Human-Computer Interaction, Optimization and Robotic Applications (ICHORA)*, IEEE, 2026. DOI: [10.1109/ICHORA69323.2026.11537199](https://doi.org/10.1109/ICHORA69323.2026.11537199)
- [30] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba e P. Abbeel, «Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World,» em *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2017, pp. 23–30.
- [31] M. Elrod, N. Mehrabi, R. Amin, M. Kaur, L. Cheng, J. Martin e A. Razi, «Graph Based Deep Reinforcement Learning Aided by Transformers for Multi-Agent Cooperation,» *arXiv preprint arXiv:2504.08195*, 2025.
- [32] A. Iskandar, H. Rostum e B. Kovács, «Using Deep Reinforcement Learning to Solve a Navigation Problem for a Swarm Robotics System,» em *2023 24th International Carpathian Control Conference (ICCC)*, IEEE, 2023, pp. 185–189.
- [33] A. Y. Ng, D. Harada e S. Russell, «Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping,» em *Proceedings of the 16th International Conference on Machine Learning (ICML)*, 1999, pp. 278–287.

[ Página intencionalmente deixada em branco. ]

## APÊNDICE A

### Configurações e Hiperparâmetros

Para garantir a reprodutibilidade integral das experiências, este apêndice reúne a configuração completa do sistema, tal como definida no ficheiro `configs/foraging.yaml` do repositório (Apêndice C). Os valores não resultaram de uma pesquisa automática de hiperparâmetros (*grid/Optuna*), mas sim de **calibração manual** informada por testes de fumo (*smoke tests*) curtos e pelas escolhas de referência da literatura e das implementações da *Stable-Baselines3*; documentam-se aqui os valores efetivamente utilizados nos treinos reportados no Capítulo 6.

TABELA A.1. Configuração do ambiente de simulação e da física.

| Parâmetro                               | Valor     | Descrição                                       |
|-----------------------------------------|-----------|-------------------------------------------------|
| <code>arena_radius</code>               | 15.0 m    | Raio da arena circular                          |
| <code>num_agents</code>                 | 20        | Número de agentes ( $N$ )                       |
| <code>num_obstacles</code>              | 100       | Obstáculos dinâmicos                            |
| <code>obstacle_radius</code>            | 0.2 m     | Raio dos obstáculos                             |
| <code>obstacle_velocity</code>          | 0.02 m/s  | Velocidade dos obstáculos                       |
| <code>nest_radius</code>                | 1.5 m     | Raio do ninho                                   |
| <code>nest_velocity</code>              | 0.015 m/s | Velocidade do ninho móvel                       |
| <code>required_to_eat</code>            | 3         | Agentes mínimos para recolha ( $k_{min}$ )      |
| <code>hunger_timer_max</code>           | 250       | Limite de fome (só Sandbox)                     |
| <code>max_steps</code>                  | 500       | Horizonte temporal base                         |
| <code>max_steps_cooperative_door</code> | 800       | Horizonte na Porta Cooperativa                  |
| <code>progress_reward_factor</code>     | 10.0      | Fator de Progresso (PBRs)                       |
| <code>geodesic_cell_size</code>         | 0.4 m     | Resolução do campo geodésico                    |
| <code>exploration_bonus</code>          | 2.0       | Bónus por célula nova                           |
| <code>exploration_cell_size</code>      | 2.0 m     | Resolução da grelha de exploração               |
| <code>min_spread_distance</code>        | 1.0 m     | Limiar da penalização de dispersão              |
| <code>spreading_penalty_factor</code>   | 1.5       | Magnitude da penalização <i>anti-clustering</i> |
| <code>agent_radius</code>               | 0.15 m    | Raio do robô                                    |
| <code>collision_penalty</code>          | -1.0      | Penalização de colisão entre agentes            |
| <code>obstacle_penalty</code>           | -2.0      | Penalização de colisão com obstáculo/parede     |
| <code>food_collected</code>             | +100.0    | Recompensa de tarefa (recolha)                  |
| <code>energy_cost</code>                | -0.05     | Custo energético por passo                      |
| <code>guillotine_threshold</code>       | -200      | Limiar da guilhotina (aos 150 passos)           |

TABELA A.2. Hiperparâmetros de treino dos três algoritmos.

| Algoritmo             | Hiperparâmetro              | Valor                          |
|-----------------------|-----------------------------|--------------------------------|
| <b>PPO</b>            | learning_rate               | $10^{-4}$                      |
|                       | n_steps (por CPU)           | 64                             |
|                       | batch_size                  | 512                            |
|                       | n_epochs                    | 4                              |
|                       | net_arch                    | [256, 256]                     |
|                       | num_cpu (SubprocVecEnv)     | 16                             |
|                       | $\gamma$ / $\lambda_{GAE}$  | 0.99 / 0.95                    |
| <b>SAC</b>            | learning_rate               | $10^{-4}$                      |
|                       | buffer_size                 | 500 000                        |
|                       | ent_coef ( $\alpha$ )       | 0.1 (fixo)                     |
|                       | net_arch                    | [256, 256]                     |
|                       | num_cpu                     | 16                             |
| <b>Evolutivo (AE)</b> | pop_size ( $K$ )            | 30                             |
|                       | mutation_rate ( $p_{mut}$ ) | 0.1                            |
|                       | sigma ( $\sigma$ )          | 0.1 ( $\times 0.995$ /geração) |
|                       | eval_episodes ( $E$ )       | 4 (seeds fixas)                |
|                       | hidden_dim ( $h$ )          | 32 ( $\approx 8k$ pesos)       |
|                       | Elitismo                    | 20% ( $e = 6$ genomas)         |

## APÊNDICE B

### Excertos de Código Relevantes

Apresentam-se os excertos que materializam os principais contributos técnicos descritos nos Capítulos 2 e seguintes. O código integral está disponível no repositório (Apêndice C).

#### B.1. Mecanismo de Atenção sobre Vizinhos (QKV)

Implementação em PyTorch puro do *forward-pass* da GNNAgent3D, sem recurso a bibliotecas de grafos. Corresponde às Equações 2.18–2.20.

```
1  # h_env: features do ego; h_all: ego + vizinhos embebidos
2  Q = self.query_proj(h_env_expanded)          # consulta (1 x d)
3  K = self.key_proj(h_all)                     # chaves (N x d)
4  V = self.value_proj(h_all)                   # valores (N x d)
5
6  # Atencao escalada por sqrt(d) -> coeficientes alpha_ij
7  scores = torch.bmm(Q, K.transpose(1, 2)) / (self.hidden_dim ** 0.5)
8  alpha = F.softmax(scores, dim=-1)
9  msg_pool = torch.bmm(alpha, V).squeeze(1)    # agregacao ponderada
10
11 # Concatena ego + mensagem agregada e gera a acao continua
12 combined = torch.cat([h_env, msg_pool], dim=1)
13 action = self.actor(combined)                 # Tanh -> [-1, 1]^3
```

LISTING B.1. Atenção de produto interno escalado sobre a vizinhança.

#### B.2. Física de Deslizamento em Muros (*Wall-Sliding*)

Em vez de imobilizar o agente, o motor projeta o movimento ortogonalmente à normal da face atingida, reproduzindo o deslizamento contínuo de um chassi móvel.

```
1  for wall in self.walls:
2      next_pos = self.agent_positions[idx] + move_global
3      delta = next_pos - wall['pos']
4      half_size = wall['size'] / 2.0
5      penetration = (half_size + self.robot_radius) - np.abs(delta)
6      if np.all(penetration > 0):                # ha colisao
7          min_axis = np.argmin(penetration)      # face atingida
8          normal = np.zeros(3)
9          normal[min_axis] = np.sign(delta[min_axis]) or 1.0
10         # remove a componente do movimento contra a normal (desliza)
11         move_global = move_global - np.dot(move_global, normal) * normal
```

LISTING B.2. Projecao ortogonal do movimento contra a normal do muro.

### B.3. Campo Geodésico para o *Reward Shaping*

Potencial de navegação  $\Phi$  usado no termo de progresso nos cenários com paredes. Uma pesquisa de custo uniforme (Dijkstra, 8-conexa) a partir da célula do ninho calcula o comprimento do caminho mais curto que contorna os obstáculos, eliminando o mínimo local da distância euclidiana. O campo é constante por cenário (paredes e ninho fixos), pelo que é calculado uma única vez e reutilizado.

```
1 # paredes dilatadas por r_robot -> caminho fisicamente percorrivél
2 dist = np.full((n, n), np.inf); dist[ni, nj] = 0.0
3 heap = [(0.0, ni, nj)]
4 while heap:
5     d, i, j = heapq.heappop(heap)
6     if d > dist[i, j]:
7         continue
8     for di, dj, cost in neigh:          # 4 ortogonais (res) + 4 diagonais
9         ii, jj = i + di, j + dj
10        if 0 <= ii < n and 0 <= jj < n and not blocked[ii, jj]:
11            nd = d + cost
12            if nd < dist[ii, jj]:
13                dist[ii, jj] = nd
14                heapq.heappush(heap, (nd, ii, jj))
15 # Phi(pos) = dist[celula(pos)]; progresso = 10.0 * (Phi_{t-1} - Phi_t)
```

LISTING B.3. Campo de distancia geodesica ao ninho (Dijkstra sobre grelha).

### B.4. Recompensa de Exploração *Count-Based*

Bónus atribuído na primeira visita a cada célula de uma grelha de discretização, complementando o termo de progresso geodésico ao premiar a cobertura de zonas novas onde o gradiente do potencial é pouco informativo.

```
1 cell = (int(pos[0] // self.exploration_cell_size),
2         int(pos[1] // self.exploration_cell_size))
3 if cell not in self.visited_cells[idx]:
4     self.visited_cells[idx].add(cell)
5     rewards[agent] += self.exploration_bonus      # +2.0 por celula nova
```

LISTING B.4. Bonus de exploracao por cobertura de celulas novas.

## APÊNDICE C

### Recursos Adicionais

A totalidade do código-fonte — simulador, implementações dos três algoritmos, *scripts* de avaliação e geração de figuras, e o *dashboard* de controlo experimental — encontra-se disponível, de forma aberta, no repositório do projeto:

<https://github.com/Pombo2001/swarm-robotics-tese>

O repositório inclui o ficheiro `configs/foraging.yaml` com a configuração reproduzida no Apêndice A, um `README.md` com o procedimento de instalação e de reprodução dos resultados (treino, avaliação determinística, testes estatísticos e geração de gráficos), e os modelos treinados por cenário. As demonstrações qualitativas do comportamento emergente são reproduzíveis localmente através do visualizador 3D (motor Ursina), executado a partir do *dashboard* de controlo.